

Data Quality as Justified Belief

Derived Controls, Deterministic Verdicts, and Evidence that Verifies Without Its Author

Ashutosh Sinha
Independent Researcher
ajsinha@gmail.com

Copyright (c) 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

Abstract

A statement that a dataset is “of good quality” is a belief about data. An organisation acts on the belief — it files a regulatory return, closes its books, prices a portfolio — and is later asked why it held it. The tools sold to answer that question run on three things that are each typed in by somebody: the controls (a person writes the SQL), the verdicts (a threshold, and increasingly a model, decides), and the record (a log that the system producing it also verifies). Each is a place where a belief can be held for no good reason while looking as though it has one.

We take the classical epistemological requirement — that knowledge be both true and *justified* — and turn it into three engineering constraints. *A control must derive from a declaration*: an owner states, in business terms, what a dataset’s grain is, what values an attribute may take, and how datasets relate, and a deterministic, total, provenance-carrying map Γ produces the controls. What no declaration covers is named by a coverage measure rather than hidden by a percentage of columns touched. *A verdict must come from a deterministic engine*: controls are written once in a rule language (PQL), lowered to one engine-neutral intermediate representation, compiled to three SQL engines and checked against an independent reference interpreter; the verdict set is larger than $\{\text{pass}, \text{fail}\}$ because a SQL screen that finds nothing has established a lower bound, not a pass; and no code path lets a model’s output decide a verdict. *Evidence must verify without trusting its author*: every judgement is appended to a hash-chained ledger with a Merkle root, and a verifier written against the standard library alone, importing nothing from the system, reaches the same conclusion.

On that base we give formal accounts of reconciliation with tolerances and timing offsets, of column lineage read from code (parsed, never executed) and the controls it can and cannot justify, of trust propagation as a choice of combining operations (only one of which is, strictly, a semiring), and of false-alarm budgets as conformal p -values with false-discovery-rate selection. Every formal claim is marked with what carries it in PRAMA, the system the author built to this account: a named test, prose, or nothing. The markers are not uniformly favourable. Two results are negative and are reported in the body: a lineage graph of value flow alone cannot see a defect that acts through a join predicate (so PRAMA now records a join as a separate, row-population edge), and the labelled defect benchmark does not yet score PRAMA itself. Section 12 counts what runs.

Contents

1	Introduction	4
1.1	The domain	4
1.2	Three places a belief is held without reason	4

1.3	The thesis	4
1.4	Contributions	5
1.5	How to read this paper, and how to check it	5
2	The objects, in engineering terms	6
3	Controls derived from declarations	6
3.1	Why derive	6
3.2	Declarations	7
3.3	The map Γ	7
3.4	Coverage, and the number that flatters	9
3.5	Proposals, and who activates	10
3.6	What Γ does not yet do	11
4	One language, one plan, a verdict that can say “not known”	12
4.1	PQL and the plan	12
4.2	The verdict set	12
4.3	Unknowns	13
4.4	A screen is not a test	13
4.5	One plan, several engines	14
4.6	Fusion	16
4.7	Where PQL stops: custom SQL and delegates	16
4.8	Numbers	17
5	Evidence that verifies without its author	17
5.1	The record	17
5.2	A verifier that shares nothing	19
5.3	Replay, and naming why an answer moved	20
6	Reconciliation	20
7	Lineage read from code	22
7.1	Parsed, never executed	22
7.2	Blast radius	23
7.3	Controls justified by lineage	23
7.4	A negative result: value lineage cannot see a join	24
8	Models author; the engine decides	25
8.1	The boundary	25
8.2	Induced controls must be able to fail	26
8.3	The gateway	26
9	Trust and false-alarm budgets	27
9.1	Trust as a choice of semiring	27
9.2	Conformal p -values	28
9.3	Selection across many monitors	29
10	Metadata, correlation and fitness for purpose	30
11	Evaluation	31
11.1	Eight case studies	31
11.2	The labelled defect benchmark	32
11.3	Measurements of the system itself	33

12 The claims register	33
13 Related work	36
14 Limitations and how this could be wrong	36
15 Conclusion	37

1 Introduction

1.1 The domain

A large bank holds its authoritative data in several hundred places: warehouses, operational databases, files landed nightly by counterparties, message streams, and the general ledger. Some of that data feeds a regulatory return whose errors are fined by the million [1]; some feeds a risk number that a board relies on; some feeds the month-end close. The Basel Committee’s principles for risk data aggregation [2] ask a bank to show that such data is accurate, complete and timely, and — the part that is expensive — to *show* it, to a supervisor, after the fact.

The tooling market for this problem is large and fragmented [13, 9]. Rule frameworks let an engineer write assertions against a table [30, 6]; observability products fit statistical monitors to column metrics; catalogues record ownership and glossary terms; reconciliation is sold separately. A research literature on constraint discovery [25, 26, 18] and error detection and repair [28, 21] addresses what an engineer might check, and a recent one addresses what a language model might propose [19, 20, 33, 35].

1.2 Three places a belief is held without reason

We read the practical failures of this tooling as three failures of justification.

The control is typed in. A person writes `SELECT count(*) ... WHERE isin IS NULL`. The control encodes a belief about what the dataset should be, but the belief is recorded nowhere except in the control. When the dataset’s grain changes, nothing connects the change to the control. When an auditor asks why a critical data element has no accuracy check, the answer is that nobody wrote one, and nothing counted the absence.

The verdict is decided by something that is not accountable. A control “passes” because a SQL screen returned zero rows, when the screen could only express the shape of an identifier and not its check digit; a freshness control “passes” because a row count was positive. Increasingly, a model is asked whether a column “looks right”. In each case a verdict is issued by a mechanism that cannot justify it.

The evidence certifies itself. The execution log lives in the same database, under the same administrator, as the system whose correctness it records. A record that can be verified only by the software that wrote it is testimony, not evidence.

1.3 The thesis

The Sanskrit *pramā* names valid cognition: knowledge that is both true and arrived at by a reliable means. A true belief held for a bad reason is not *pramā*. We think this is the right standard for a data quality claim, and that it decomposes into three engineering constraints, each checkable.

J1. A control derives from a declaration. Every control either is produced by a total, deterministic map Γ from something an accountable owner declared, or carries the provenance of the proposal that a named person accepted. What is declared but not controlled, and what is critical but not declared, is measured.

J2. A verdict comes from a deterministic engine. A verdict is a function of the compiled plan and the data, computed the same way on every engine the plan targets, and never a function of a model’s output. Where the engine cannot decide, the verdict says so.

J3. Evidence verifies without its author. The record of every verdict can be checked by a program that shares no code with the system that produced it.

In plain terms. “Declare it. Prove it. Trust it.” A business owner says what the data means; the system turns that into checks nobody had to write; a deterministic engine runs them and records what it found in a form that anyone can check with a short script. A model may help write a check or explain a result. It never decides whether the data passed.

1.4 Contributions

1. A formal account of *declaration-derived controls*: the map Γ from a business semantic layer (grain, rhythm, value domains, semantic types, and thirteen relationship kinds) to controls, with determinism, identity stability, provenance and refusal properties, and a coverage measure over (attribute, dimension) pairs (Section 3).
2. A verdict semantics for an engine-neutral rule language that separates an exact test from a *screen*, makes **indeterminate** first class, and is checked across three SQL engines and a reference interpreter; and the fusion of controls into shared scans (Section 4).
3. An evidence ledger whose verification is independent of the system that writes it (Section 5).
4. A tolerance and timing semantics for reconciliation, with a counting result for timing offsets and its measurement on a month-end close (Section 6).
5. An account of column lineage read from code, of the controls it justifies and the ones it must hold back, a negative result about what value lineage cannot see, and the population edge that repairs it (Section 7).
6. The boundary “models author, the engine decides”, enforced by an architectural test with a counterfactual, by a gate that refuses a control that cannot fail, and by an evaluation-gated model gateway with a hash-chained call ledger (Section 8).
7. Trust propagation as a choice of semiring, and false-alarm budgets as conformal p -values with Benjamini–Hochberg selection (Section 9); metadata-implied rules, correlation and fitness search (Section 10).
8. An evaluation that reports eight planted-defect case studies, both columns, and a labelled benchmark whose bounds and ablations were run and whose comparison with PRAMA was not (Section 11).

1.5 How to read this paper, and how to check it

Each definition, proposition and theorem is followed by a marker: **Runs**: names a test in the repository, as `path::test_name`, that exercises the claim; **Runs**: followed by prose names a mechanism without a single test; **In part**: says what is weaker in the code than on the page; **Not executed**: marks mathematics that constrains the system without the system checking it; and **Not in PRAMA**: marks a construction that is not built. Every cited test was located in the repository at the time of writing. A reader with the repository can run any of them with `pytest -q '<path>::<name>'`. Numbers are quoted only where they come from a run we performed or from an assertion in a named test, and each says which.

The system is PRAMA version 0.1.0 (`src/prama/version.py`), about 115,000 lines of Python under `src/prama` and 249 test modules under `tests`. It is proprietary; the paper describes it and does not release it.

2 The objects, in engineering terms

Before the formal material, the vocabulary in the terms an engineer uses.

Estate

Everything a tenant governs: connectors to sources, and the datasets read through them.

Dataset

A table, a view, a feed of files, or a set of feeds, with an owner, a criticality tier, a *grain* (“one row per trade”) and a *rhythm* (“arrives by 06:30 on business days”).

Attribute

A column given a business meaning: optional or mandatory, a semantic type such as ISIN or LEI, a value domain (a code list or a range), and possibly the mark of a *critical data element* (CDE).

Relationship

A typed, declared link between two datasets: *references*, *reconciles-with*, *derives-from*, and ten others (Table 2).

Control

An executable check, written in PQL, with a severity, a quality dimension and a threshold. A control has a stable *identity* and a content *hash*.

Proposal

A candidate control, from any origin, awaiting a person. Proposals are the only way a control enters the estate.

Plan

The engine-neutral intermediate representation (IR) a control lowers to; the thing that is compiled, fused, executed and hashed.

Verdict

The outcome of executing a plan: *pass*, *fail*, *indeterminate*, *error* or *skipped*.

Evidence record

An append-only, hash-linked record of one verdict and what it was computed from.

The pipeline is: *declare* $\rightarrow \Gamma$ *derives* proposals $\rightarrow$ a person *activates* $\rightarrow$ the plan is *compiled* and *fused* $\rightarrow$ an engine *executes* $\rightarrow$ the verdict is *judged* from metrics $\rightarrow$ the evidence is *recorded* and can be *verified* and *replayed*. Models sit beside this pipeline — they may draft a declaration, a proposal or an explanation — and never inside the step from metrics to verdict.

3 Controls derived from declarations

3.1 Why derive

A control written by hand restates a belief about a dataset in a second place. The first place is wherever the belief really lives: in the owner’s head, in a data dictionary, in a regulatory instruction. Two statements of one rule diverge, and they diverge silently, because only one of them is executed. The remedy we propose is not better discipline in writing controls but a change in what is written: the owner writes the belief once, in business terms, and the control is computed from it. When the belief changes, the control changes, because it is the same object seen twice.

3.2 Declarations

Definition 3.1 (Dataset declaration). A *dataset declaration* is a tuple

$$d = (ref, G, K, \rho, \theta, A, \alpha, t)$$

where *ref* names a dataset; *G* is a *grain*, a set of attribute names whose values identify a row; *K* is an optional business key; *ρ* is a *rhythm* (an arrival expectation with a calendar, and an expected volume); *θ* is a temporality; *A* is a finite set of *attribute declarations*; *α* is an optional authoritativeness statement (“replica of *X*”); and *t* is a criticality tier. An attribute declaration $a \in A$ carries an optionality, an optional semantic type (e.g. ISIN, LEI, ISO date, monetary amount with its currency column), an optional value domain (a code list or a range), and a flag marking it a critical data element. A declaration records who declared it and when.

Definition 3.2 (Relationship declaration). A *relationship declaration* is a tuple

$$r = (k, left, right, M, c, \epsilon, \delta)$$

where *k* is one of the thirteen kinds of Table 2, *M* is a match key (pairs of attributes), *c* an optional compared attribute, *ε* an optional tolerance and *δ* an optional timing offset. A relationship is either *proposed* (discovered from data or lineage) or *confirmed* by a person.

Runs: tests/semantic/test_services.py::TestRelationshipService::test_every_kind_is_offered_in_business_language

Runs: tests/semantic/test_services.py::TestRelationshipService::test_a_discovered_relationship_is_never_confirmed_automatically

3.3 The map Γ

Definition 3.3 (Control, identity and hash). A *control* is a PQL statement together with a severity, a quality dimension, and a *provenance*. Its *identity* is a function $\iota(declaration, rule, subject)$ of where it came from and never of its text. Its *content hash* is a function of its text. Identity answers “is this the same control?”; the hash answers “has it changed?”.

Definition 3.4 (Γ). For a dataset declaration *d*, $\Gamma(d)$ is obtained by applying, in a fixed order, the rules γ_{grain} , γ_{key} , γ_{rhythm} , $\gamma_{temporality}$, γ_{attr} and γ_{auth} , each of which returns a finite set of controls and a finite set of *unsatisfiable* findings, and then *coalescing*: controls whose checks are equal are merged into one control whose rule is the sorted join of the contributing rules. For a relationship *r*, $\Gamma(r)$ dispatches on the kind *k* and returns a lineage edge together with either controls, comparison specifications for the reconciliation engine, or a refusal naming what is missing.

Table 1 lists what each dataset rule produces and Table 2 what each relationship kind produces. The second table is honest about a gap between what a kind *declares* it generates and what the code *emits*: every kind emits something, but several emit fewer control families than the design corpus lists.

Proposition 3.5 (Determinism and identity stability). Γ is a function: regenerating an unchanged declaration yields the same identities and the same hashes. Editing a declaration so that a control’s check changes yields a control with the same identity and a different hash, so the control is updated rather than orphaned and replaced.

Proof. Each rule is a pure function of the declaration, applied in a fixed order, and coalescing sorts its rule names, so the output is independent of iteration order. Identity is computed from $(declaration, rule, subject)$, none of which an edit to a threshold or a code list changes; the hash is computed from the text, which such an edit does change. $\square$

Declaration	Control emitted by Γ
Grain G	HAS UNIQUE KEY over G (grain.uniqueness); completeness of each member of G
Business key K	uniqueness over K
Rhythm ρ	a freshness control with the declared calendar; a volume control; seasonality deferred to a monitor
Temporality θ	a temporal control
Mandatory attribute	IS NOT NULL
Semantic type	IS VALID '<type>', naming its standard
Value domain	IN CODELIST or a range
Monetary amount	IN CODELIST 'iso4217' on the currency column beside it
Authoritativeness	parity with the authoritative source

Table 1: The dataset rules of Γ (src/prama/derive/generator.py).

Runs: tests/derive/test_generator.py::test_regenerating_an_unchanged_declaration_changes_nothing

Runs: tests/derive/test_generator.py::test_editing_a_declaration_updates_a_control_rather_than_orphaning_it

Runs: tests/propose/test_end_to_end.py::test_a_second_run_of_the_generator_adds_nothing

Proposition 3.6 (Provenance). *Every control in $\Gamma(d)$ carries a provenance naming its origin (declaration), the rule that produced it, the declaration and attribute it came from, the declaring person and the time. The question “why does this control exist?” therefore has an answer that is data, not recollection.*

Runs: tests/derive/test_generator.py::test_every_generated_control_can_say_why_it_exists

Proposition 3.7 (Round trip). *Every generated control is valid PQL: for every $c \in \Gamma(d)$, $\text{render}(\text{parse}(c)) = c$.*

Runs: tests/derive/test_generator.py::test_every_generated_control_re_parses_to_itself

Proposition 3.8 (Coalescing). *If two rules produce the same check, Γ emits one control, whose rule records both.*

Runs: tests/derive/test_generator.py::test_two_rules_reaching_the_same_check_produce_one_control

Proposition 3.9 (Refusal, not omission). *Γ is total in the following sense: every declaration either produces a control or produces a finding that says why it did not. A grain naming a column that does not exist is reported, not emitted; a reconciliation with no tolerance is refused with the reason; every one of the thirteen relationship kinds yields an edge and either an output or a refusal.*

Runs: tests/derive/test_generator.py::test_a_grain_naming_a_column_that_does_not_exist_is_reported_not_emitted

Runs: tests/derive/test_relationships.py::test_a_reconciliation_with_no_tolerance_is_refused_with_the_reason

Runs: tests/derive/test_relationships.py::test_every_kind_produces_something_and_nothing_silently

Runs: tests/derive/test_relationships.py::test_feeds_generates_an_edge_and_deliberately_no_control

Kind	Declared families	Actually emitted
references	referential integrity, orphans, key coverage	PQL REFERENCES (a correlated EXISTS)
reconciles-with derives-from	reconciliation, breaks, certificate aggregate parity, trust edge, impact	a reconciliation specification aggregate parity
feeds mirrors	lineage edge, arrival chain, latency row-count, content parity, staleness	an edge, and deliberately no control row-count parity, value parity (if compared), staleness
aggregates enriches	roll-up parity, trust edge enrichment coverage	aggregate parity PQL reference check, completeness dimension
supersedes same-entity-as temporal-successor parent-of	migration parity, dual run resolution, duplicates, identifiers roll-forward hierarchy completeness, cycles, orphans	value parity identifier consistency roll-forward PQL orphan-node check only; no cycle detection
mutually-exclusive together-complete	overlap population completeness	overlap coverage

Table 2: The thirteen relationship kinds (`src/prama/derive/relationships.py`). Only three kinds yield PQL controls; the rest yield comparison specifications executed by the reconciliation engine.

In plain terms. Writing “one row per trade, identified by trade id” produces a uniqueness check and a not-null check on the trade id. Writing “this column is an ISIN” produces a validity check that names ISO 6166. Nobody writes SQL. If the owner changes the rule, the same control is updated. If a statement cannot be turned into a check — a grain naming a column that is not there — the system says so instead of quietly dropping it.

Worked example: a trading book

Case study 1 (`case-studies/01-trading-book-sqlite`) declares six datasets over one SQLite database, with 5,000 trades and nine planted defects. In our run, Γ produced 65 controls and reported three declarations that produced none, each with a reason: “*Prama has no validator or code list called ‘currency’*”. Executed, the 65 controls gave 44 passes, 5 failures and 16 not established. The five failures are exactly the five planted defects that a single-dataset declaration can see: the duplicated positions (grain), 60 missing notionals (a mandatory CDE), 7 negative quantities (a range), and the currency ‘EURO’ (a code list, reported twice, as validity and consistency). The four it did not find are each named in the run’s output with the reason: an ISIN checksum (a screen, Section 4.4), an orphan instrument and a ledger adjustment (both need a *relationship*, which this study does not declare), and a settlement date before the trade date — for which no control exists because nothing in the declaration says how the two dates relate. That last one is a gap in the declaration, not in Γ , and the study leaves it in to show the difference.

3.4 Coverage, and the number that flatters

A percentage of columns “with a control” flatters: a single `IS NOT NULL` makes a column covered while its format, domain and accuracy go unchecked.

Definition 3.10 (Applicable pairs and coverage). For a declared dataset, the *applicable pairs* P are the (attribute, dimension) pairs, over the dimensions completeness, validity, consistency and accuracy, that the declaration makes meaningful: validity only where a semantic type or a value domain is declared; consistency only for an amount with a currency; accuracy only for a

CDE; completeness except for an optional attribute. Uniqueness and timeliness are counted at dataset level. The *coverage* is $\kappa = |P_{\text{cov}}|/|P|$, where P_{cov} are the pairs some active control addresses. The *touched fraction* τ is the share of attributes with at least one control.

Proposition 3.11 (The honest and the flattering number). *τ can equal 1 while $\kappa < 1/2$. Both are reported, side by side, and every uncovered pair is reported with the declaration or control that would close it, ranked by a risk weight that favours CDEs and obligations.*

Proof. Take attributes each with a mandatory flag, a semantic type and a CDE mark, and one IS NOT NULL control on each. Then $\tau = 1$, while of the three applicable pairs per attribute only completeness is covered, so $\kappa = 1/3$. $\square$

Runs: tests/derive/test_coverage.py::test_coverage_is_measured_per_dimension_not_per_column

Runs: tests/derive/test_coverage.py::test_the_flattering_number_is_reported_beside_the_honest_one

Runs: tests/derive/test_coverage.py::test_a_dimension_an_attribute_cannot_have_is_not_counted_against_it

Runs: tests/derive/test_coverage.py::test_every_gap_names_what_would_close_it

Proposition 3.12 (A dataset cannot vouch for its own accuracy). *No dataset declaration alone covers the accuracy pair of a CDE. Coverage of a CDE’s accuracy requires a relationship declaration.*

Proof. The accuracy dimension is satisfied only by a control comparing the attribute with something outside its row — in PRAMA, a REFERENCES control or a comparison. The dataset rules of Table 1 emit neither; only the relationship rules of Table 2 do. $\square$

Runs: tests/propose/test_wave6_acceptance.py::test_a_dataset_declaration_alone_cannot_cover_its_own_cdes

Runs: tests/propose/test_wave6_acceptance.py::test_adding_the_relationships_reaches_the_target

In plain terms. Accuracy means “agrees with something else”. A dataset, however carefully described, cannot be compared with nothing. That is a small piece of logic, but it is the reason relationships — a line drawn between two datasets — carry the most weight in PRAMA’s estate-maturity score.

The implementation roadmap reports the effect on a realistic Tier-1 declaration: the dataset declaration alone reaches 82% of pairs and 71% of CDE pairs, and adding the relationship declarations reaches 100% of both. The test asserts the weaker statement that relationships reach at least 95% of CDE pairs and 80% overall; the 82/71 figures are prose and we did not reproduce them.

In part: the proposition runs; the specific percentages are stated in docs/19, not asserted.

3.5 Proposals, and who activates

Definition 3.13 (Proposal and origin). A *proposal* is a candidate control with an *origin* in the totally ordered set `declaration > document > import > mining > example > induction` (the last is a language model), a backtest against current data, and a utility score. Proposals are the only channel through which a control enters the estate.

Proposition 3.14 (Only a declaration may skip review, and not always). *A proposal may be marked as not needing review only if its origin is **declaration**, and not even then if its backtest violates more than 20% of rows (a declaration that would “drown the queue” is probably wrong about the data). Mining, documents, examples and model induction always need review.*

Runs: tests/derive/test_provenance.py::test_only_a_declaration_may_run_without_review

Runs: tests/propose/test_queue.py::test_not_even_a_declaration_may_run_if_it_would_drown_the_queue

Runs: tests/induce/test_inducer.py::test_a_good_answer_becomes_an_induced_control

Remark 3.15. Reading the code closely, the “may skip review” mark is counted in the queue summary and used nowhere else: the only activation path is `ControlDAO.activate`, which requires an `approved_by` identity and writes it into the control’s provenance. In practice, therefore, a person activates every control, including a declared one; the case studies do so explicitly. The generator’s docstring (“nothing it emits is active”) is correct; the `may_auto_activate` property overstates what happens.

Runs: src/prama/db/dao/control.py (activate requires `approved_by`); no test asserts the absence of another activation path.

Proposition 3.16 (Corroboration merges). *When two origins propose a control with the same identity, the result is one proposal, held by the stronger origin and marked corroborated, whose confidence rises by a bounded amount. A rejected proposal is not offered again unless the evidence changes materially.*

Runs: tests/propose/test_queue.py::test_two_origins_reaching_the_same_rule_merge_rather_than_duplicate

Runs: tests/propose/test_queue.py::test_the_stronger_origin_keeps_the_proposal

Runs: tests/propose/test_wave6_acceptance.py::test_mining_and_declaring_the_same_rule_produces_one_corroborated_proposal

Runs: tests/propose/test_utility.py::test_corroboration_raises_confidence_modestly

Runs: tests/propose/test_queue.py::test_a_rejected_proposal_is_not_offered_again

The utility score is a fixed linear combination of criticality, coverage gap, confidence and an acceptance prior learned from past decisions, Laplace-smoothed and capped so that the learned term cannot dominate. Tier-1 datasets require a second person (maker-checker).

Runs: tests/propose/test_utility.py::test_the_learned_term_cannot_dominate_the_ranking

Runs: tests/semantic/test_services.py::TestApprovalPolicy::test_tier_one_requires_a_second_person

3.6 What Γ does not yet do

Declaring a rhythm generates a freshness control, and that control has no execution strategy on a plain table: it can never reach a verdict. The repository pins this as a *strict* expected failure, so the day it is fixed the test fails and forces the finding to be closed. In case study 1 the stale JPY rate is reported “not established” for this reason. Freshness is established in PRAMA only where there is an arrival record — a feed of files, as in case study 2, where the missing and the duplicated deliveries are both found by the arrival ledger rather than by SQL. In part: tests/derive/test_generator.py::test_a_generated_freshness_control_can_reach_a_verdict is a strict xfail.

The design corpus lists roll-forward, aggregate parity and row-count parity as *PQL* forms. They exist as comparison specifications produced by Γ ; the PQL surface for them is still a specification.

Not in PRAMA: PQL syntax for roll-forward, aggregate and row-count parity; cycle detection for parent-of.

4 One language, one plan, a verdict that can say “not known”

4.1 PQL and the plan

PQL is PRAMA’s control language. A control is a **CHECK** over a dataset — a predicate over rows (`amount >= 0, ccy IN CODELIST 'iso4217', isin IS VALID 'isin'`), a dataset property (`HAS UNIQUE KEY (trade_id), REFERENCES`), a named Python delegate, or a single read-only custom query — with a threshold (`AT MOST 5 ROWS, BELOW 0.1%`), a severity, a dimension, an optional scope (`WHERE, FOR EACH`) and a policy for unknowns. A **RECONCILE** statement compares two datasets (Section 6).

```
CHECK trades.notional IS NOT NULL SEVERITY critical DIMENSION completeness
CHECK trades.isin IS VALID 'isin'
CHECK trades HAS UNIQUE KEY (trade_id)
CHECK trades.ccy IN CODELIST 'iso4217'
      TREAT UNKNOWN AS PASS BECAUSE 'optional on internal moves'
```

Definition 4.1 (Plan and plan identity). A control *lowers* to a *plan*: a typed, engine-neutral logical form comprising an assertion kind, a predicate, an unknown policy, a scope, the metrics to compute and a threshold, together with any code-list values and validator implementations it names. Its *meaning* $\mu(P)$ is that tuple and its IR version; its identity is $plan_id(P) = ir:sha256:H(\text{canon}(\mu(P)))$, where *canon* serialises with sorted keys, no whitespace and UTF-8. Severity, description, rationale and provenance are not part of μ .

Proposition 4.2 (What the plan identity closes over). *Changing a code list changes the plan identity of every control that names it; a plan that needs a residual validator hashes differently from the same plan without one; and the identity is the same whichever entry point lowered the control.*

Runs: `tests/backend/test_worked_example.py::TestTheCodelistIsClosedOver::test_changing_the_codelist_changes_the_control`

Runs: `tests/backend/test_two_stage.py::test_a_two_stage_plan_hashes_differently_from_a_screen_only_one`

Runs: `tests/backend/test_resolve.py::TestItDoesNotChangeAnythingElse::test_a_control_with_no_codelist_hashes_identically`

In part: no test asserts that severity or provenance are *excluded* from the hash; it is true of `ControlPlan.meaning` by inspection.

In plain terms. A control’s fingerprint changes when what it checks changes — including the list of allowed values it points at, and including the Python behind a validator it names — and does not change when somebody edits its description. Last month’s evidence therefore refers to exactly the control that produced it.

4.2 The verdict set

Definition 4.3 (Verdicts). $V = \{\text{pass}, \text{fail}, \text{indeterminate}, \text{error}, \text{skipped}\}$. Given the metrics m an engine returned for a plan with threshold θ :

$$\text{judge}(m, \theta) = \begin{cases} \text{indeterminate} & \text{if the metric } \theta \text{ reads is absent,} \\ & \text{or } m.\text{scanned_rows} \text{ is absent or 0,} \\ & \text{or } \theta \text{ is relative and its denominator is 0;} \\ \text{pass} & \text{if } \theta \text{ holds of } m; \\ \text{fail} & \text{otherwise.} \end{cases}$$

error records an execution failure (an unreadable source, an absent delegate host) as a finding rather than a crash. A segmented control fails if any segment fails, and is indeterminate if any segment is indeterminate and none fails. The actionable verdicts are `{fail, error, indeterminate}`.

Proposition 4.4 (Nothing scanned is not a pass). *A control over an empty scope is indeterminate, for absolute and relative thresholds alike. The exception is a row-count control, which is a claim about the count itself: over an empty table it fails.*

Runs: `tests/backend/test_reference.py::TestScopes::test_an_empty_scope_is_indeterminate_not_a_pass`

Runs: `tests/backend/test_two_stage.py::TestNothingScannedIsNotAPass::test_an_absolute_threshold_refuses_an_empty_scan`

Runs: `tests/backend/test_two_stage.py::TestNothingScannedIsNotAPass::test_the_two_agree`

Runs: `tests/backend/test_two_stage.py::TestNothingScannedIsNotAPass::test_a_row_count_control_still_judges_an_empty_table`

Runs: `tests/delegates/test_delegates.py::test_without_a_delegate_host_the_control_is_an_error_not_a_skip`

4.3 Unknowns

SQL evaluates a comparison with a null to `unknown`, and a `WHERE` clause drops unknown rows. A control written as “count the rows where the predicate is false” therefore silently passes every null.

Definition 4.5 (Unknown policy). For a row predicate φ with three-valued semantics, the violation count under policy π is

$$v_{\pi} = |\{x : \varphi(x) = \text{false}\}| + [\pi = \text{violation}] \cdot |\{x : \varphi(x) = \text{unknown}\}|.$$

The default is $\pi = \text{violation}$. `TREAT UNKNOWN AS PASS` selects the other policy and must carry a `BECAUSE` clause stating why.

Proposition 4.6 (The default is the conservative one). $v_{\text{pass}} \leq v_{\text{violation}}$, with equality exactly when no row evaluates to unknown. Hence for any upper-bound threshold, a control that passes under the default also passes under `TREAT UNKNOWN AS PASS`, and not conversely.

Proof. Immediate from Definition 4.5; the second term is non-negative. An upper-bound threshold is monotone in v . $\square$

Runs: `tests/pql/test_parser.py::TestUnknownsAreViolationsByDefault::test_an_unknown_counts_against_the_control`

Runs: `tests/pql/test_parser.py::TestUnknownsAreViolationsByDefault::test_but_only_with_a_reason`

Runs: `tests/backend/test_engine_conformance.py::TestTheCorpusDiscriminates::test_the_unknown_policy_changes_the_count`

Runs: `tests/backend/test_reference.py::TestUnknownsAreViolations::test_a_null_counts_against_the_control`

4.4 A screen is not a test

Some validity rules have no faithful SQL form. An LEI’s check digits are computed by ISO 7064 MOD 97-10 over a letter-to-number mapping; an ISIN’s by a Luhn variant. SQL can check the *shape* (length, alphabet, position of digits) but not the arithmetic, portably. The tempting move is to compile the shape check and call it the control. That control passes every well-shaped fabricated identifier.

Definition 4.7 (Screen and residual). For an exact validator V over values, a *screen* is a predicate S with $V(x) \Rightarrow S(x)$ for every x : a necessary condition every valid value satisfies, and which invalid values may also satisfy. The *residual* is V itself, run over the rows the screen admits. A plan whose SQL computes only S is *incomplete*.

Proposition 4.8 (Screen soundness). Let $n_S = |\{x : \neg S(x)\}|$ and $n_V = |\{x : \neg V(x)\}|$ over a scope. Then $n_S \leq n_V$. Hence an incomplete plan may soundly report *fail* when n_S breaches the threshold, and may not report *pass* when $n_S = 0$: the only sound verdict for an incomplete plan with n_S within the threshold is *indeterminate*.

Proof. By contraposition of $V \Rightarrow S$, $\neg S(x) \Rightarrow \neg V(x)$, so the screen’s violations are a subset of the validator’s. A breach by the subset is a breach by the whole, for any upper-bound threshold. Conversely $n_S = 0$ is consistent with any $n_V \geq 0$, so it cannot establish a threshold on n_V . $\square$

Runs: tests/backend/test_two_stage.py::test_a_valid_value_is_never_rejected_by_the_screen

Runs: tests/backend/test_two_stage.py::test_the_screen_alone_would_have_passed_the_fabricated_value

Runs: tests/backend/test_two_stage.py::test_the_compiled_query_declares_that_it_is_not_the_whole_test

Runs: tests/execute/test_control_run.py::TestAnIncompleteScreenCannotPass::test_zero_violations_from_a_screen_is_not_a_pass

Runs: tests/execute/test_control_run.py::TestAnIncompleteScreenCannotPass::test_a_screen_that_found_violations_still_fails

Worked example: a fabricated LEI

A twenty-character value with the right alphabet and wrong check digits passes the screen: the SQL counts $n_S = 0$. The exact validator counts $n_V = 1$. PRAMA compiles the screen, marks the query incomplete, and on $n_S = 0$ reports “a pass cannot be reported from a screen alone”. Across the eight case studies this is the most frequent “not established” line: in case study 1, ten controls on ISIN, LEI and ISO-date columns are reported this way, including the one on the column where twelve ISINs with wrong checksums were planted.

In plain terms. Zero rows failing a rough check is not the same as zero rows failing the real check. Reporting it as a pass would be the most expensive lie a data quality tool can tell, because it is told about exactly the columns whose correctness SQL cannot express.

The residual is not run on the SQL path in the case studies: the run reports “residual not run”. This is by design — the residual is a Python validator and runs where rows are available — but it means that on a pure pushdown deployment the checksum columns stay *indeterminate* until a residual executor is configured.

In part: screens are sound and never report a pass; the residual executes in the reference interpreter, not as part of an SQL pushdown run.

4.5 One plan, several engines

PRAMA compiles plans to three SQL dialects — PostgreSQL, DuckDB and SQLite — and executes them with a fourth, independent evaluator: a reference interpreter in Python that is never given SQL and shares no code with the compiler.

Definition 4.9 (Conformance). A backend e *conforms* on a corpus case c if it returns the verdict, the metrics and the per-segment breakdown that the case declares, or if it refuses c at compile time with a named reason. A refusal is a conforming outcome; a silently different answer is not.

Proposition 4.10 (Cross-engine agreement on the corpus). *Every case of the conformance corpus yields the same verdict and metrics on every engine that runs it, and each case is compared by at least two genuinely different engines. The corpus discriminates: it contains passing and failing cases, a case where the unknown policy changes the count, a duplicate key that counts once, a segment that fails alone, and rates judged against the rows actually scanned after a filter.*

Runs: tests/backend/test_engine_conformance.py::TestTheEnginesAgree::test_every_case_gives_the_same_answer_everywhere

Runs: tests/backend/test_engine_conformance.py::TestTheEnginesAgree::test_at_least_two_genuinely_different_engines_took_part

Runs: tests/backend/test_engine_conformance.py::TestTheCorpusDiscriminates::test_a_rate_is_judged_against_the_rows_actually_scanned

Runs: tests/backend/test_reference.py::TestItSharesNothingWithTheCompiler::test_it_never_asks_for_sql

The corpus has 25 cases (src/prama/backend/corpus.py). DuckDB, SQLite and the reference interpreter always run; PostgreSQL runs when PRAMA_TEST_POSTGRES_DSN is set. We ran the backend suite both ways: without PostgreSQL, and against a PostgreSQL 16 container, where all 138 backend tests passed with none skipped.

Beyond the corpus, a seeded generator produces random controls (250 by default) and requires every engine to agree on each; the generated set is required to exercise all three of **pass**, **fail** and **indeterminate**, and every generated control is reproducible from its seed.

Runs: tests/backend/test_generated_equivalence.py::TestTheEnginesAgreeOnAllOfThem::test_no_generated_control_produces_a_disagreement

Runs: tests/backend/test_generated_equivalence.py::TestTheEnginesAgreeOnAllOfThem::test_the_generated_set_exercises_all_three_outcomes

Runs: tests/backend/test_generated_equivalence.py::TestEveryGeneratedControlIsValid::test_they_are_reproducible_from_their_seed

Remark 4.11. This is seeded differential testing, not property-based testing with shrinking, and it is not a proof of semantic equivalence. Full equivalence across engines is probably unattainable for regular-expression flavours, collations and decimal semantics; the response is a portable subset and compile-time refusal.

Not executed: a semantic-equivalence proof between the compiler and the reference interpreter.

Proposition 4.12 (Refusal is conformance). *A construct an engine cannot express is refused at compile time with a named reason, never approximated. SQLite refuses approximate distinct counts and sampling, and a key wider than its column limit; it does regular expressions through a function the executor registers, and the corpus confirms it agrees with DuckDB. A fused group containing a refused control is stopped, not silently shortened.*

Runs: tests/backend/test_engine_conformance.py::TestRefusalIsConformance::test_sqlite_refuses_what_it_genuinely_cannot_do

Runs: tests/backend/test_engine_conformance.py::TestRefusalIsConformance::test_sqlite_does_regex_through_a_registered_function

Runs: tests/pql/test_deep_input.py::test_a_500_column_key_runs_where_the_engine_can_and_is_refused_where_it_cannot

Runs: tests/backend/test_fuse.py::TestTheFusedQuery::test_a_control_the_engine_cannot_run_stops_that_group

In part: DuckDB’s refusal of back-references and look-arounds (RE2) is implemented and untested.

In plain terms. The same control means the same thing on every database it runs on, and a fourth, deliberately separate implementation checks that. Where a database genuinely cannot do something, the control refuses to run there and says why; it never runs a weaker version and reports the answer as if it were the same.

Building the case studies against real data found two defects of exactly this kind, both since fixed and recorded in `case-studies/README.md`: a rounding function that double-rounded on DuckDB (a bare `CAST(x AS NUMERIC)` is `DECIMAL(18,3)` there), producing 131 false one-cent breaks in 3,000 rows; and an unknown function name that compiled straight through to SQL while the reference interpreter returned unknown. The second is the more instructive: the independent check existed, and the two disagreed silently because nothing compared them on that input. There is now a function catalogue in which no function can exist without both a lowering and a reference implementation.

4.6 Fusion

A source pays for scans, not for controls.

Definition 4.13 (Fusion). Plans over the same dataset and scope are grouped into one query that computes every plan’s metrics in a single pass, computing identical metrics once. The cost of a suite is the number of scans.

Proposition 4.14 (Fusion preserves answers). *Every control in a fused query receives the metrics, and hence the verdict, it receives when run alone.*

Proof sketch. Each plan’s metrics are aggregate expressions over the same scope, conditioned on the plan’s own predicate (e.g. a `SUM(CASE WHEN ...)`); aggregates over one scan are independent of the other aggregates computed in it. Plans whose scopes differ are not fused. $\square$

Runs: `tests/backend/test_fuse.py::TestTheFusedQuery::test_it_answers_every_control_identically`

Runs: `tests/soak/test_throughput.py::TestFusionActuallySaves::test_fused_answers_match_unfused_ones_at_volume`

Runs: `tests/backend/test_fuse.py::TestCost::test_cost_is_counted_in_scans_not_controls`

The unit suite runs nine controls in three scans. At volume, the soak test asserts that a realistic generated suite needs at most half the naive number of scans and that the saving grows with the suite; in our run it printed “*400 controls in 160 scans = 40.0% of a naive pass*”.

Runs: `tests/soak/test_throughput.py::TestFusionActuallySaves::test_a_realistic_suite_needs_far_fewer_scans`

Runs: `tests/soak/test_throughput.py::TestFusionActuallySaves::test_the_saving_grows_with_the_suite`

4.7 Where PQL stops: custom SQL and delegates

Two escape hatches exist, and both are shaped to keep the verdict with the engine.

`CHECK CUSTOM SQL` accepts exactly one read-only query, checked at parse time, and runs only on the engines it names. There is deliberately no inline Python.

Runs: `tests/pql/test_custom_sql.py::test_anything_but_one_read_only_query_is_refused_at_parse_time`

Runs: `tests/pql/test_custom_sql.py::test_it_runs_on_an_engine_it_names_and_is_refused_on_others`

Definition 4.15 (Delegate). A *delegate* is a registered Python class, named in PQL as `CHECK ds USING DELEGATE 'acme.settlement_cycle01' (...) BELOW 0.1%`, that returns *counts* — rows scanned and rows violating — and never a verdict. The control’s threshold judges the counts. A delegate’s source hash is folded into the evidence.

Proposition 4.16 (A delegate cannot decide). *The verdict of a delegate control is $\text{judge}(m, \theta)$ of Definition 4.3 applied to the delegate’s counts, so a delegate that establishes nothing yields *indeterminate*, not *pass*. A delegate importing a network, clock or model client is refused before it is imported; one that is not deterministic on admission probes is refused; a file changed after admission is refused; input over the row ceiling is refused, not truncated; and a sandboxed run agrees with an in-process run.*

Runs: `tests/delegates/test_delegates.py::test_a_result_that_establishes_nothing_is_indeterminate_not_a_pass`

Runs: `tests/delegates/test_delegates.py::test_an_impure_file_is_refused_before_it_is_imported`

Runs: `tests/delegates/test_delegates.py::test_admission_refuses_what_cannot_replay_or_be_judged`

Runs: `tests/delegates/test_streaming_and_testkit.py::test_a_file_changed_after_admission_is_refused_in_the_sandbox`

Runs: `tests/delegates/test_delegates.py::test_too_many_rows_are_refused_not_truncated`

Runs: `tests/delegates/test_delegates.py::test_sandboxed_and_in_process_runs_agree_and_both_see_json_values`

The sandbox is a subprocess with CPU and memory limits and a wall-clock timeout. It is weaker than its description in three respects: the worker inherits the parent’s environment, there is no network namespace (purity is enforced by a static scan of imports, not by the operating system), and neither the timeout nor the memory limit is exercised by a test.

In part: process isolation and static vetting run; OS-level network isolation is absent; limits untested.

Worked example: a check PQL cannot say

Case study 5 plants 23 US trades booked on T+2 after the move to T+1, 9 EU trades settling on a TARGET2 closing day, and 4 unreadable dates. The column comparison `settlement_date >= trade_date` passes all of them, because every one settles after it trades. A bank-owned delegate that knows each market’s cycle and calendar returns 36 violations (23 late, 9 early, 4 unreadable); the threshold, not the delegate, turns them into a failure. A second delegate, run on a remote agent inside a payments zone, applies Benford’s first-digit test and fails the planted ledger (MAD 0.0176, above Nigrini’s 0.015 nonconformity band) while keeping the example rows inside the zone.

4.8 Numbers

Operator arithmetic in the function library and the reference interpreter is exact decimal: one tenth plus two tenths is three tenths, and division by zero is unknown rather than an error. Threshold values are parsed and stored as binary floating point, and survive rendering digit for digit; the comparison of a metric with a threshold is therefore a float comparison.

Runs: `tests/pql/test_numbers_survive.py::TestOperatorArithmeticIsExact::test_a_tenth_plus_two_tenths_is_three_tenths`

Runs: `tests/pql/test_numbers_survive.py::TestRenderingKeepsEveryDigit::test_a_threshold_survives_a_round_trip`

In part: thresholds are float, not decimal.

5 Evidence that verifies without its author

5.1 The record

Definition 5.1 (Evidence record and chain). An *evidence record* e_i is a mapping of fields — tenant, control identity and version, plan identity, snapshot, verdict, metrics, dimensions, a reference by hash to any sample rows, the format version, the sequence number i — together with three hashes. Its *content hash* is $h_i = H(\text{canon}(e_i \setminus \{\text{prev}, h, rh\}))$, computed over

exactly the fields its own format version defines. Its *record hash* is $rh_i = H(rh_{i-1} \parallel h_i)$, with $rh_0 = \text{GENESIS} = 0^{64}$. H is SHA-256 and canon is canonical JSON (keys sorted at every level, no whitespace, UTF-8). A chain is valid if its sequence numbers are contiguous from 1, every h_i recomputes, and every rh_i links.

Theorem 5.2 (Tamper evidence). *Suppose a verifier holds an authentic chain head rh_n . If H is collision resistant, then any alteration of a field of any e_i ($i \leq n$), any deletion, insertion or reordering of records, or any truncation of the chain, is detected by the validity check, except with negligible probability.*

Proof sketch. An altered field changes $\text{canon}(e_i)$ and hence, except on a collision, h_i ; if h_i is left unchanged the recomputation fails. If h_i is recomputed, rh_i changes, and by induction every subsequent record hash changes, so rh_n differs from the authentic head unless a collision was found at some step. Deletion, insertion and reordering break contiguity of sequence numbers or the link equation at the point of change, and a truncated chain ends at a head different from rh_n . $\square$

```
Runs: tests/evidence/test_ledger.py::TestTheChain::test_a_fresh_chain_starts_from_genesis
Runs: tests/evidence/test_ledger.py::TestTheChain::test_each_record_links_to_the_one_before
Runs: tests/evidence/test_ledger.py::TestTheChain::test_a_missing_record_is_detected_as_a_gap
Runs: tests/evidence/test_ledger.py::TestTheChain::test_reordering_is_detected
Runs: tests/evidence/test_independent_verifier.py::TestItRejectsTampering::test_a_truncated_
file_is_caught
Runs: tests/evidence/test_independent_verifier.py::TestItRejectsTampering::test_a_removed_
middle_record_is_caught
Runs: tests/evidence/test_independent_verifier.py::TestItRejectsTampering::test_a_reordered_
pair_is_caught
```

The theorem is only as strong as the authenticity of the head. PRAMA signs the head with HMAC-SHA256, which proves nothing to a party that does not hold the key — the code says so in as many words — and seals exported bundles with an HMAC and, optionally, an Ed25519 signature over the manifest that verifies with the public key alone. A bundle without a signature is reported untrustworthy even when every hash matches.

```
Runs: tests/evidence/test_ledger.py::TestSigning::test_a_different_key_does_not_verify
Runs: tests/evidence/test_ledger.py::TestSigning::test_a_changed_head_does_not_verify
Runs: tests/security/test_bundle.py::TestAsymmetricSignature::test_a_signature_verifies_with_
only_the_public_half
Runs: tests/security/test_bundle.py::TestAnUnsignedBundleIsNotABundle::test_no_signature_is_
not_trustworthy_even_when_the_hashes_match
```

Not in PRAMA: publication of chain heads to an external timestamping service or transparency log [16, 17]. Without it, an operator holding the HMAC key could rewrite a chain and re-sign its head; the Ed25519 bundle signature binds only what was exported.

Definition 5.3 (Merkle root). For record hashes $x_1, \dots, x_n$, the root is computed level by level, combining adjacent pairs with $H(x_{2j-1} \parallel x_{2j})$; an unpaired node at the end of a level is *promoted* unchanged to the next level; the root of the empty set is GENESIS [22].

Remark 5.4. Promotion rather than duplication matters: the common alternative, duplicating the last node, gives the lists $[a, b, c]$ and $[a, b, c, c]$ the same root, which lets a duplicated record pass unnoticed.

```
Runs: tests/evidence/test_ledger.py::TestMerkleRoots::test_an_odd_node_is_promoted_not_
duplicated
Runs: tests/evidence/test_ledger.py::TestMerkleRoots::test_an_empty_set_has_no_root
```

5.2 A verifier that shares nothing

Proposition 5.5 (Independent verification). *The script `scripts/verify_evidence.py` imports only the Python standard library, runs where PRAMA is not importable, and checks content hashes, links, contiguity, erasure seals, the manifest’s record count and file digest, the Merkle root and the chain head. On every input tested, including records with non-ASCII text, an added field, and doctored manifests, it reaches the same conclusion as PRAMA’s own verifier. A second, test-local re-implementation written from the prose description of the algorithm also agrees.*

```
Runs: tests/evidence/test_independent_verifier.py::TestItIsGenuinelyIndependent::test_it_
imports_only_the_standard_library
```

```
Runs: tests/evidence/test_independent_verifier.py::TestItIsGenuinelyIndependent::test_it_runs_
without_prama_importable
```

```
Runs: tests/evidence/test_independent_verifier.py::TestTheTwoVerifiersAgree::test_a_non_ascii_
record_verifies_in_both
```

```
Runs: tests/evidence/test_independent_verifier.py::TestTheTwoVerifiersAgree::test_an_added_
field_is_a_breach_in_both
```

```
Runs: tests/evidence/test_independent_verifier.py::TestItRejectsTampering::test_a_doctored_
manifest_is_caught
```

```
Runs: tests/evidence/test_ledger.py::TestVerifiableWithoutPrama::test_an_independent_reader_
reaches_the_same_conclusion
```

```
Runs: tests/evidence/test_ledger.py::TestVerifiableWithoutPrama::test_and_the_same_conclusion_
when_a_record_is_altered
```

```
Runs: tests/evidence/test_end_to_end.py::TestVerifyingWithoutPrama::test_an_exported_chain_
checks_with_stdlib_alone
```

In plain terms. An auditor does not have to trust PRAMA to check PRAMA’s records. A short script that uses nothing but Python’s standard library reads the exported file and says whether anything was changed, removed or reordered. The two verifiers once disagreed (QA finding C4, since fixed); that disagreement was a defect in the evidence layer, it was found by running the two against each other, and a test — including a record with non-ASCII text — now holds them together.

Erasure without breaking the chain. A record that must be erased (a legal hold ends, or a subject exercises a right to erasure) is replaced by a tombstone that keeps the original content hash, so the chain still links, and carries a seal of its own over who erased it and why. Rewriting who erased a record is caught.

```
Runs: tests/evidence/test_independent_verifier.py::TestAnErasureIsItselfTamperEvident::test_
rewriting_who_erased_it_is_caught
```

```
Runs: tests/evidence/test_independent_verifier.py::TestAnErasureIsItselfTamperEvident::test_
the_chain_still_verifies_across_the_erasure
```

Format versions. A field added to the record format is hashed only in records whose version defines it, so a chain mixing old and new records verifies.

```
Runs: tests/evidence/test_format_versions.py::TestOldRecordsSurviveANewBuild::test_a_mixed_
chain_verifies
```

Size. Sample rows are referenced by hash rather than carried. The tests assert that the median record is under 2 KB; the roadmap reports 848 bytes on the worked example, a figure we did not reproduce.

```
Runs: tests/evidence/test_ledger.py::TestSize::test_a_record_is_well_under_two_kilobytes
```

```
Runs: tests/evidence/test_ledger.py::TestSize::test_samples_are_referenced_not_carried
```

5.3 Replay, and naming why an answer moved

Definition 5.6 (Replay outcome). Re-executing the plan of a record yields one of: *identical*; *stable* (the answer held while the inputs moved); one of six named causes of divergence — *data_changed*, *control_changed*, *engine_changed*, *parameters_changed*, *snapshot_not_exact*, *coverage_changed*; or *unexplained*. Causes are tried in a fixed precedence (a changed control explains more than changed data), and *unexplained* is never folded into an ordinary cause.

Proposition 5.7 (Replay classifies). *Each named cause is assigned when, and only when, its condition holds in the tested scenarios; a changed control takes precedence over changed data; an engine disagreement is escalated; and a run whose answer held is reported as **stable**, not diverged, so that a nightly replay against fresh data does not bury the divergences that matter.*

Runs: tests/evidence/test_replay.py::TestNamingTheCause::test_moved_data_is_the_ordinary_case

Runs: tests/evidence/test_replay.py::TestNamingTheCause::test_an_edited_control_makes_the_answers_incomparable

Runs: tests/evidence/test_replay.py::TestNamingTheCause::test_the_control_wins_when_both_the_control_and_the_data_moved

Runs: tests/evidence/test_replay.py::TestNamingTheCause::test_an_engine_disagreement_is_escalated

Runs: tests/evidence/test_replay.py::TestNamingTheCause::test_an_inexact_snapshot_explains_itself

Runs: tests/evidence/test_replay.py::TestNamingTheCause::test_an_unexplained_divergence_is_reported_as_alarming

Runs: tests/execute/test_watermarks.py::TestTheWidthTravelsWithTheVerdict::test_a_replay_at_a_different_width_is_not_blamed_on_the_data

Runs: tests/evidence/test_end_to_end.py::TestReplay::test_the_controls_whose_answer_held_are_not_reported_as_diverged

Every case study ends by verifying its chain and printing its Merkle root; all eight verified in our runs, over 10 to 81 records each.

6 Reconciliation

Reconciliation — showing that two systems that should agree do — is the most expensive data quality failure in banking, and is usually sold as a separate product. In PRAMA it is a PQL statement:

```
RECONCILE subledger AGAINST general_ledger
  ON (account, cost_centre, posting_date)
  COMPARING amount = balance_eur WITHIN 0.01 EUR
  NORMALISING currency TO 'EUR' USING RATES fx_rates
  OFFSET BY 1 DAY
```

Definition 6.1 (Reconciliation). A reconciliation $\mathcal{R} = (L, R, M, (c_L, c_R), \epsilon, \nu, k)$ has a left and a right dataset, a match key M , compared attributes, a tolerance $\epsilon = (a, r)$ with absolute bound $a \geq 0$ and relative bound $r \geq 0$ (either may be absent, treated as 0), an optional normalisation ν converting the left amount into the right’s currency through a declared rate table, and an optional offset of k days on one date component of the key.

Definition 6.2 (Tolerance). A difference Δ against a magnitude m is *permitted* when $|\Delta| \leq a \vee |\Delta| \leq r|m|$ — “whichever is larger”. A pair is a *break* when the difference is not permitted, that is, when it breaches *both* bounds.

Proposition 6.3 (Tolerance laws). (i) *Permission is monotone: enlarging a or r never creates a break.* (ii) *For large positions the relative bound governs; for small ones the absolute bound.* (iii) *Against a zero magnitude, a purely relative bound permits only an exact match.* (iv) *A difference breaching both bounds is a break however large either bound is individually.*

Proof. (i) Each disjunct is monotone in its bound. (ii) $r|m| > a$ exactly when $|m| > a/r$. (iii) $r \cdot 0 = 0$. (iv) is Definition 6.2 restated. $\square$

Runs: tests/recon/test_reconciliation.py::TestWhicheverIsLarger::test_the_relative_bound_governs_a_large_position

Runs: tests/recon/test_reconciliation.py::TestWhicheverIsLarger::test_a_difference_breaching_both_is_still_a_break

Runs: tests/recon/test_reconciliation.py::TestWhicheverIsLarger::test_a_relative_bound_against_nothing_permits_only_an_exact_match

Runs: tests/recon/test_reconcile_pql.py::test_the_engine_counts_genuine_breaks_and_the_threshold_judges

Runs: tests/recon/test_reconcile_pql.py::test_normalising_converts_before_comparing

Definition 6.4 (Matching with an offset). Rows are first matched exactly on M . Only left rows still unmatched are then probed at offsets $\pm 1, \dots, \pm k$ days on the named date component of M (by default the last), against right rows also still unmatched.

Lemma 6.5 (An offset never steals an exact match). *No row that has an exact match is paired across an offset.*

Proof. Exact matching runs to completion before any offset probe, and probes consider only rows left unmatched by it. $\square$

Runs: tests/recon/test_reconciliation.py::test_an_exact_match_is_never_quietly_paired_with_the_wrong_day

Runs: tests/recon/test_reconciliation.py::test_a_date_one_day_apart_is_matched_rather_than_double_counted

Proposition 6.6 (The price of ignoring timing). *Let T be a set of keys whose right-hand entry is posted $\ell \leq k$ days late, such that for each $t \in T$ neither the original date nor the late date has another unmatched counterpart and the amounts agree within tolerance. Let B_k be the number of breaks with offset k and B_0 the number without. Then*

$$B_0 = B_k + 2|T|.$$

Proof. With the offset, by Lemma 6.5 and the hypothesis, each $t \in T$ is paired across the offset and contributes no break. Without it, t has a left row at date d with no right row at d (a *missing*) and a right row at $d + \ell$ with no left row there (an *extra*): two breaks. Breaks not involving T are unaffected, since exact matching is the same in both runs. $\square$

Runs: case study 6, run by us: 5 breaks with OFFSET BY 1 DAY, 9 without, with $|T| = 2$. No test asserts either number; run.py prints them and README.md states them.

Worked example: a month-end close

Case study 6 builds an ERP with 840 subledger rows, 280 ledger rows and three currencies, and plants four defects: a manual journal of 250.00 EUR posted to the ledger only, a day's entries that never reached the ledger, a ledger balance on an account the subledger never booked, and two entries from the last evening that reached the ledger the next morning. The reconciliation above found five breaks needing a person — three *genuine* (the journal, on three dates), one *missing*,

one *extra* — and no break for the timing difference. Run without `OFFSET BY 1 DAY`, the same books show nine: each of the two late entries becomes one missing on the last day and one extra the next, exactly as Proposition 6.6 predicts.

Definition 6.7 (Break classes). Each break is classified as `timing`, `fx`, `rounding`, `missing`, `extra`, `duplicate`, `sign` or `genuine`. Only `timing` is expected to clear by itself; `sign`, `duplicate` and `fx` are configuration faults and are routed away from the data steward; the violation count of a reconciliation is the number of breaks that need a person.

Runs: `tests/recon/test_reconciliation.py::test_a_timing_difference_is_expected_to_clear`

Runs: `tests/recon/test_reconciliation.py::test_a_sign_convention_error_is_named_rather_than_reported_as_a_big_break`

Runs: `tests/recon/test_reconciliation.py::test_a_duplicated_posting_is_named`

Runs: `tests/recon/test_reconciliation.py::test_configuration_faults_are_routed_away_from_the_data_steward`

The statement round-trips through the parser and renders as a sentence. A filter is written after the dataset it applies to, `RECONCILE a WHERE p AGAINST b WHERE q`, and a single `WHERE` on the whole control is refused: a filter on one side only reports every row it dropped from that side as missing on the other. An unfiltered reconciliation’s plan is unchanged by the addition, so evidence already written still names it.

Runs: `tests/recon/test_reconcile_pql.py::test_it_round_trips_and_says_what_it_checks`

Runs: `tests/recon/test_reconcile_pql.py::test_a_filter_on_the_whole_control_is_refused`

Runs: `tests/recon/test_reconcile_pql.py::test_each_side_filters_its_own_rows`

Runs: `tests/lineage/test_joins.py::test_a_filtered_copy_reconciles_against_the_same_rows_of_its_source`

Not in PRAMA: user-written `CLASSIFY` rules on `RECONCILE` (the engine’s built-in classification is used); a signed reconciliation certificate from the break workbench.

7 Lineage read from code

7.1 Parsed, never executed

Definition 7.1 (Lineage graph). A *column lineage graph* is a directed graph whose nodes are qualified columns and whose edges carry a transform τ , one of `identity`, `rename`, `derived`, `aggregated`, `filter` and `join_key`; a status, `parsed` or `inferred`; a confidence; and the source span it was read from.

PRAMA reads SQL with `sqlglot`, falling back to a pattern reader that records the fallback as a gap; PySpark, pandas and Airflow code are read with Python’s `ast` module (importing a DAG file runs it, so it is never imported); Power BI models are read as JSON. An edge produced by a parser is `parsed` with confidence 1.0; an edge produced by the fallback is `inferred` at 0.8; an edge proposed by a language model is `inferred` and capped at 0.85, and is kept only if it is checked against the code.

Runs: `tests/lineage/test_pyspark.py::test_the_code_is_parsed_never_run`

Runs: `tests/codeintake/test_intake.py::test_nothing_received_is_executable`

Runs: `tests/lineage/test_sql.py::test_the_parser_path_is_the_one_running`

Runs: `tests/lineage/test_sql.py::test_what_the_parser_cannot_read_falls_back_and_says_so`

Runs: `tests/lineage/test_sql.py::test_a_where_clause_column_feeds_the_target_as_a_filter`

Runs: `tests/codeintake/test_model_lineage.py::test_the_model_pass_keeps_only_checked_edges_as_inferred`

On a gold fixture ETL repository the extractor is required to reach precision at least 0.98 and recall at least 0.95 on value edges, with filter and join-key edges excluded from the measure.
Runs: `tests/codeintake/test_gold_fixture.py::test_bankco_etl_meets_the_gate`

7.2 Blast radius

Definition 7.2 (Impact). Each transform has an attenuation $\alpha(\tau) \in (0, 1]$: identity and rename 1.0, filter 0.9, join key 0.8, derived 0.7, aggregated 0.35. The impact of a path is the product of its attenuations, and the *impact* of a defect at s on a node v is the maximum over paths from s to v of length at most 12, reported when it is at least 0.01.

Proposition 7.3 (Strongest path). *Because every $\alpha \leq 1$, extending a path never increases its impact. A breadth-first search that re-expands a node whenever a strictly stronger path reaches it (a label-correcting search) therefore terminates, even on a cyclic graph, with every node at its maximum-product impact; the first path found is not in general the strongest.*

Runs: `tests/lineage/test_graph.py::test_the_strongest_path_wins_rather_than_the_first_one_found`

Runs: `tests/lineage/test_graph.py::test_an_aggregate_attenuates_more_than_a_copy`

Runs: `tests/lineage/test_graph.py::test_a_filter_is_barely_attenuated`

Runs: `tests/lineage/test_graph.py::test_a_negligible_impact_is_counted_rather_than_listed`

The attenuations are declared constants, not estimates. They order transforms sensibly and they carry no statistical meaning; a reader should read “35% of the defect” as a ranking, not a measurement. The code also contains a comment saying a filter “is not attenuated at all” beside the value 0.9; the value is what runs.

Not executed: the attenuation constants are not calibrated against observed defect propagation.

7.3 Controls justified by lineage

A lineage edge is a claim about how one column is computed from another. Some controls follow from that claim, and some do not.

Definition 7.4 (Lineage-derived proposals). From a lineage graph and the active controls, PRAMA proposes: *propagation*, carrying a row-level control on s to t when t is an identity or rename copy of s ; *referential*, a REFERENCES check from a copied key column to its source; and *reconciliation*, a RECONCILE t AGAINST s ... WITHIN 0 when keys and an amount are copied at the same grain. A proposal is *held*, with its reason, when any edge it rests on is *inferred*, or — for a reconciliation — when a *filter* edge lies between source and target.

Proposition 7.5 (What a copy justifies). *Let t be produced from s by selecting a subset of rows and copying a column by identity or rename. (i) For any row predicate φ , if every row of s satisfies φ then every row of t does; so propagating a row-level control from s to t is sound, filtered or not. (ii) A reconciliation of t against s on their keys reports every row the filter removed as missing, so it is sound only when no filter lies between them. (iii) Neither (i) nor (ii) holds for an aggregated column.*

Proof. (i) Every row of t is a copy of a row of s . (ii) Rows of s outside the filter have no counterpart in t . (iii) An aggregate is not a copy of any row, so a row-level predicate on the input says nothing about the output, and the output’s grain differs. $\square$

Runs: `tests/derive/test_lineage_controls.py::test_a_control_is_carried_to_a_copied_column`

Runs: `tests/derive/test_lineage_controls.py::test_an_aggregated_column_does_not_inherit_a_row_level_control`

```
Runs: tests/derive/test_lineage_controls.py::test_a_copied_key_must_exist_in_its_source
Runs: tests/derive/test_lineage_controls.py::test_a_proposal_on_an_inferred_edge_is_held
Runs: tests/derive/test_lineage_controls.py::test_every_proposal_is_valid_pql
Runs: tests/recon/test_reconcile_pql.py::test_lineage_proposes_a_reconciliation_for_a_copy_at_the_same_grain
```

In plain terms. If staging is a straight copy of the raw feed, a rule that must hold on every raw row must hold on every staged row too, so PRAMA proposes the same rule downstream. But if staging keeps only booked trades, reconciling it against the raw feed would report every cancelled trade as a break, so that proposal is held back with the reason. A guess about lineage never becomes a control without somebody confirming the guess first.

Case study 8 found both halves of Proposition 7.5(ii) the hard way: the first run proposed a reconciliation across the filter and reported 118 breaks, one per cancelled trade, and the proposed reconciliation had no tolerance and could not run. Both are fixed. The proposal says `WITHIN 0`, and the filter edge now carries its condition when it is over one table, so the proposal applies the same filter to the source side (`AGAINST raw.trades WHERE status = 'BOOKED'`); a condition that cannot be carried over holds the proposal, with the reason.

```
Runs: tests/recon/test_reconcile_pql.py::test_lineage_proposes_a_reconciliation_for_a_copy_at_the_same_grain
```

7.4 A negative result: value lineage cannot see a join

Proposition 7.6 (Join predicates are invisible to value lineage). *Let m be a column of a view $Q = \pi_{...m...}(S \bowtie_{S.c=F.c} F)$ and suppose c is not projected into Q . In a lineage graph whose edges record only value flow (identity, rename, derived, aggregated) and filters on `WHERE` columns, there is no path from $S.c$ to $Q.m$; yet a defect in $S.c$ that makes the join predicate false removes rows from Q , and so changes every aggregate of m .*

Proof. No value of $S.c$ is copied, transformed or aggregated into any column of Q , and c is not in a `WHERE` clause, so the extractor emits no edge from $S.c$ into Q . The dependence of Q on $S.c$ is a dependence of Q 's row population, not of any column's values. $\square$

Runs: case study 8, run by us. No test asserts it; the nearest test is `tests/lineage/test_pyspark.py::test_a_join_is_a_gap_not_a_guess`, which requires the PySpark reader to record a join as a gap rather than invent an edge.

Worked example: four lower-case currencies, 605 million of notional

Case study 8 builds a warehouse by running an ETL repository: raw feed, staging (booked trades only), and a mart that joins staging to an FX table on currency and aggregates exposure, with a Power BI model on top. Intake read three files and, when the study was first built, emitted twelve parsed column edges. Two defects are planted in the raw feed: five notionals carrying the sign of the side, and four currencies in lower case. The owner writes two controls on the raw columns; lineage proposes the rest. The notional's blast radius runs to the dashboard — 100% at staging, 35% at the mart's exposure (aggregated), 35% at the dashboard's exposure (renamed), 12% at its total (0.35^2) — and the propagated control finds the five at staging too. The currency's blast radius *stops at staging*: the mart uses currency only in the join. Yet its effect there is real, and the run measures it: three staged trades with no FX rate drop out of the mart, 605,000,000 of notional with them, with no error, no null and no count anywhere that changes. Only the control on the raw column saw it.

The repair: a population edge. The proposition is about a graph of value flow, so the repair is not a better value edge but a different kind of node. The SQL reader now records each equality in an inner or left join’s `ON` clause as two `join_key` edges, one from each side’s key, into the view’s *rows*, written $Q.*$ — the same node a `WHERE` column already fed — and reads a common table expression through to its real table. The blast radius then treats $Q.*$ as a population node: a change in which rows exist reaches every column computed over them, and the path says so (*computed over these rows*). The edge does not pretend to be a value effect: nothing flows from $S.c$ into $Q.m$ except through $Q.*$, and a reader sees the rows named as the thing affected. What remains a heuristic is its weight, 0.8, which is declared rather than estimated like every other attenuation (Section 7).

Runs: `tests/lineage/test_joins.py::test_a_join_key_is_recorded_through_a_cte_to_the_real_table`

Runs: `tests/lineage/test_joins.py::test_a_cte_is_not_mistaken_for_a_table`

Runs: `tests/lineage/test_joins.py::test_the_blast_radius_follows_a_join_to_the_dashboard`

The join also justifies a control, by the same argument as Proposition 7.5: in $S \bowtie_{S.c=F.c} F$ every S row whose c has no match in F is dropped by an inner join, or reaches Q with nothing joined to it by a left join, and in neither case does anything fail. So lineage proposes `CHECK S.c REFERENCES F.c DIMENSION integrity` on the driving side, held while the edge is inferred.

Runs: `tests/recon/test_reconcile_pql.py::test_a_join_proposes_that_every_driving_row_finds_its_match`

Rerun with both changes, case study 8 reads fourteen edges. The proposed join check fails on 3 of 1,882 staged trades, which is where the defect happens and not only where it entered, and the currency’s blast radius now runs from staging to the mart’s rows (80%), the mart’s exposure (80%), the dashboard’s exposure (80%) and its total (28%). The filtered reconciliation of staging against the raw feed passes: the copy is faithful, and the defects are in what it faithfully copied. We still take the result to mean that controls belong at the source; lineage carries them downstream, and now carries them across a join.

Not in PRAMA: join-key edges from the PySpark reader, which records a join as a gap; population edges from `EXISTS` and `IN` subqueries.

8 Models author; the engine decides

8.1 The boundary

Language models are good at drafting a rule from a data dictionary, at ranking datasets for a question put in plain words, and at explaining a failure. They are not accountable for a verdict, and their output is not reproducible in the sense evidence requires. PRAMA’s rule is that a model may *author*, *rank*, *explain*, *calibrate* and *summarise*, and that no code path lets a model’s output determine whether data passed.

Definition 8.1 (Model-free verdict path). The *verdict path* is the sequence `parse` $\rightarrow$ `type-check` $\rightarrow$ `lower` $\rightarrow$ `compile` $\rightarrow$ `execute` $\rightarrow$ `judge` $\rightarrow$ `record`. It is *model-free* if no function on it takes as input a value returned by a model client.

Proposition 8.2 (What the architectural guard establishes). *The architecture test fails the build if any source module both refers to a model (by names such as `llm`, `openai`, `completion`, `build_prompt`) and produces or assigns a verdict (`verdict`, `pass_fail`, `is_violation`, `assert_outcome`), with docstrings stripped before scanning. A counterfactual module that calls a model and returns a verdict makes the guard fire. The guard is a syntactic necessary condition for Definition 8.1, not a proof of it: a model’s output laundered through a neutral name in a separate module would not be caught.*

Runs: tests/architecture/test_layering.py::TestModelVerdicts::test_no_module_both_calls_a_model_and_produces_a_verdict

Runs: tests/architecture/test_layering.py::TestModelVerdicts::test_the_guard_still_fires_on_a_module_that_would_break_the_rule

Runs: tests/architecture/test_layering.py::TestModelVerdicts::test_the_guard_ignores_prose_about_the_rule

Runs: tests/architecture/test_layering.py::TestModelVerdicts::test_the_exemption_is_a_ban_list_not_a_model_call

Not executed: an information-flow proof of Definition 8.1; the guard approximates it by co-occurrence.

The structural argument is stronger than the guard. A verdict is $\text{judge}(m, \theta)$ (Definition 4.3), where m comes from an engine executing a compiled plan and θ from the plan itself; a plan comes from parsing PQL text that a person activated (Section 3.5). A model can influence a verdict only by authoring PQL that a person then activates, at which point the verdict is a function of that person’s decision and the data, and the evidence names both. Validators and delegates that import a model client are refused at admission (Proposition 4.16).

8.2 Induced controls must be able to fail

A model asked for a rule can return one that is well formed, type-correct, compiles, runs — and cannot fail: `CHECK trades.side MATCHES /.*/`, or a bound of `qty >= -1e308`.

Definition 8.3 (Validation gates). An induced control is *validated* only after five gates: it parses; it type-checks against the catalogue; it compiles; it runs in a sandbox over at most 2,000 sample rows; and a *counterfactual* gate finds data on which it fails. The type `Validated` cannot be constructed unless all five passed.

Proposition 8.4 (No vacuous control reaches a reviewer). *A control that accepts every value it could be shown is rejected at the counterfactual gate, however well formed, and never enters the proposal queue.*

Runs: tests/induce/test_validate.py::test_a_validated_control_cannot_be_built_without_passing_every_gate

Runs: tests/induce/test_validate.py::test_a_control_that_cannot_fail_is_rejected_however_well_formed

Runs: tests/induce/test_validate.py::test_a_bound_so_wide_that_nothing_breaches_it_is_rejected

Runs: tests/induce/test_inducer.py::test_a_vacuous_answer_never_reaches_a_reviewer

In plain terms. A check that cannot fail is worth nothing, and a model is quite capable of writing one that looks reasonable. So before anybody is shown a model’s suggestion, PRAMA tries to make it fail. If it cannot, the suggestion is thrown away.

8.3 The gateway

All model calls go through one gateway. A caller asks for a *purpose* (author, explain, discover, embed), never for a model; a profile maps each purpose to an ordered route of providers. A purpose with no profile is routed to a mock provider that answers with nothing — so a fresh installation uses no model at all, and still records the call.

Definition 8.5 (Call ledger). Every call is appended to a ledger whose entries are sealed as in Definition 5.1: each seal is a hash over the entry and the previous seal. Payloads are redacted by default (secrets, card numbers, IBANs) and may expire; an expired payload is blanked and the chain still verifies.

Runs: tests/llm/test_gateway.py::test_the_seal_chains

Runs: tests/llm/test_governance.py::test_an_expired_payload_is_blanked_and_the_chain_still_verifies

Runs: tests/llm/test_governance.py::test_payloads_follow_policy_and_are_redacted_by_default

Runs: tests/llm/test_gateway.py::test_an_unconfigured_purpose_falls_to_the_mock_which_answers_nothing

In part: prama llm verify is not itself exercised by a test; the method it calls is.

Proposition 8.6 (Evaluation gates activation). *A prompt template version is approved only after a passing evaluation run, and not by its author; a new profile version cannot be activated without its own passing run; and with no model configured no evaluation can pass. Graders are deterministic.*

Runs: tests/llm/test_governance.py::test_a_template_version_is_approved_only_after_a_passing_run_and_by_another

Runs: tests/llm/test_governance.py::test_with_the_gate_a_new_profile_version_waits_for_its_own_passing_run

Runs: tests/llm/test_governance.py::test_with_no_model_configured_nothing_can_pass

Personal data may not be sent to a hosted model; when no provider on the route may receive a payload, the refusal is recorded and raised; a spent budget refuses with HTTP 429.

Runs: tests/llm/test_providers.py::test_pii_may_not_be_sent_to_a_hosted_model

Runs: tests/llm/test_gateway.py::test_when_nothing_may_receive_it_the_refusal_is_recorded_and_raised

Runs: tests/llm/test_providers.py::TestTheThreeDefects::test_a_secret_and_a_card_number_are_withheld_from_the_prompt

Runs: tests/llm/test_providers.py::TestPatternRedaction::test_a_valid_iban_is_withheld_and_a_lookalike_is_not

Runs: tests/api/test_llm_api.py::test_a_spent_budget_refuses_with_429

Not in PRAMA: any measurement of the *quality* of model-authored rules — acceptance rate, precision, or the comparison of mining, induction and their fusion. The mechanism for merging corroborated origins runs (Proposition 3.16); the claim that fusion beats either alone is about reviewers' decisions and was not measured.

9 Trust and false-alarm budgets

9.1 Trust as a choice of semiring

A score for a derived column should depend on the scores of what it is derived from. How it should depend is a modelling choice, and the choice is algebraic.

Definition 9.1 (Trust propagation). Given a lineage DAG, base scores $s(v) \in [0, 1]$, and a pair of operations $(\otimes, \oplus)$ for combining along a path and across paths, the propagated trust of v combines its own score with $\bigoplus_{\text{paths } p \rightarrow v} \bigotimes_{u \in p} s(u)$. PRAMA offers four choices:

Name	along a path ($\otimes$)	across paths ($\oplus$)
all inputs matter (default)	product	min
redundant sources	product	max
weakest link	min	min
product-mean	product	mean

Attenuation (Definition 7.2) applies to the *deficit* $1 - s$, not to the score: an aggregate passes on only part of a source's defect.

Remark 9.2. The code calls all four “semirings”. Strictly, only one is. *Redundant sources*, $([0, 1], \max, \times, 0, 1)$, is the Viterbi semiring of the provenance literature [15]. The default, $(\min, \times)$, has both operations associative and commutative and $\times$ distributes over $\min$ on $[0, 1]$, but the identity of $\min$ is 1, which does not annihilate under $\times$; the same holds for *weakest link*, $(\min, \min)$. *Product–mean* is not associative across paths at all, since a mean of means over unequal groups is not the mean. Nothing in PRAMA relies on annihilation — a node with no inbound path is simply scored on its own evidence — so the gap is one of naming, and we record it rather than repair it here. The choice is exposed rather than hidden because the choices genuinely disagree on the same graph.

Runs: `tests/score/test_trust.py::test_the_default_treats_inputs_as_complementary`

Runs: `tests/score/test_trust.py::test_the_semirings_genuinely_disagree`

Runs: `tests/score/test_trust.py::test_a_copy_inherits_the_defect_intact`

Runs: `tests/score/test_trust.py::test_an_aggregate_makes_a_column_more_trustworthy_than_its_source`

Runs: `tests/score/test_trust.py::test_a_control_that_caught_the_problem_stops_it_propagating`

Not executed: the algebraic laws (associativity, distributivity) of each choice; the tests check behaviour on examples, not the laws.

9.2 Conformal p -values

A statistical monitor raises an alert when an observation is unusual. “Unusual” needs a false-alarm rate, and a rate needs a guarantee.

Definition 9.3 (Conformal p -value). Given calibration scores $s_1, \dots, s_n$ and a new score s , the conformal p -value is

$$p(s) = \frac{1 + |\{i : s_i \geq s\}|}{n + 1}.$$

An alert is raised at level α when $p(s) \leq \alpha$.

Theorem 9.4 (Validity [34]). *If $s_1, \dots, s_n, s$ are exchangeable, then $\Pr[p(s) \leq \alpha] \leq \alpha$ for every $\alpha \in [0, 1]$.*

Proof sketch. Under exchangeability the rank of s among the $n + 1$ scores is uniform over $\{1, \dots, n + 1\}$ (breaking ties conservatively), and $p(s) \leq \alpha$ requires s to rank among the top $\lfloor \alpha(n + 1) \rfloor$. $\square$

Runs: `tests/calibrate/test_conformal.py::test_a_normal_observation_alerts_at_most_alpha_of_the_time`

Runs: `tests/calibrate/test_conformal.py::test_the_guarantee_survives_a_distribution_no_z_score_would`

Runs: `tests/calibrate/test_conformal.py::test_the_plus_one_is_the_guarantee_and_not_rounding`

Corollary 9.5 (A budget the history cannot express). *With n calibration points the smallest attainable p -value is $1/(n + 1)$. A budget of b false alarms over N runs asks for $\alpha = b/N$; if $b/N < 1/(n + 1)$ no threshold honours it, and PRAMA refuses the budget with the arithmetic rather than silently alerting at a higher rate.*

Runs: `tests/calibrate/test_select.py::test_a_business_budget_becomes_a_statistical_level`

Exchangeability fails under drift. PRAMA offers three responses and is explicit about each one’s price: *recency weighting* (a weighted p -value with a half-life, which is no longer exact, says so, and reports its reduced effective sample size); *adaptive levels* in the style of adaptive

Regime	plain	seasonal	weighted	adaptive	changepoint
stationary	0.0040	0.0152	0.0096	0.0016	0.0040
seasonal	0.0672	0.0232	0.0696	0.0152	0.0672
level shift	0.1112	0.0904	0.0208	0.0168	0.0784
regime switch	0.0112	0.0520	0.0136	0.0080	0.0112
bursty	0.0104	0.0192	0.0080	0.0064	0.0104

Table 3: Calibration error (mean absolute gap between nominal and realised alert rate over the levels 0.01, 0.02, 0.05, 0.10 and 0.20) by drift regime and mechanism, on synthetic generators at the default 500 observations per regime. Printed by `tests/monitor/test_benchmark.py` in our run; the target is 0.02.

conformal inference [14], $\alpha_{t+1} = \alpha_t + \gamma(\alpha - \text{err}_t)$ with $\gamma = 0.02$, which controls the long-run alert rate rather than each test; and *conditioning* on a declared seasonal calendar. Adaptation cannot help when every p -value sits at its floor, and the code detects that state rather than adapting forever.

Runs: `tests/calibrate/test_conformal.py::test_a_weighted_p_value_does_not_claim_to_be_exact`

Runs: `tests/calibrate/test_conformal.py::test_recency_weighting_costs_resolution_and_the_cost_is_reported`

Runs: `tests/calibrate/test_conformal.py::test_the_long_run_alert_rate_converges_to_the_budget_under_drift`

Runs: `tests/calibrate/test_conformal.py::test_adapting_cannot_help_when_every_p_value_is_at_its_floor`

Table 3 is the result as a grid rather than a number, because no single mechanism is the answer and one number would hide that. Conditioning on season repairs seasonality and cannot repair a level shift; forgetting (weighting) nearly repairs a level shift and makes seasonality worse; in this run the adaptive level meets the target in every regime. What the test asserts is weaker than the table: that every regime has *some* mechanism meeting the target, that every mechanism is calibrated on the stationary regime, and that plain conformal fails on seasonal data.

Runs: `tests/monitor/test_benchmark.py::test_every_regime_has_a_mechanism_that_holds_the_promise`

Runs: `tests/monitor/test_benchmark.py::test_the_stationary_case_is_calibrated_by_every_mechanism`

Runs: `tests/monitor/test_benchmark.py::test_plain_conformal_fails_on_seasonal_data`

End to end, a monitor declaring a budget of 0.05 realised 0.0467 over 300 judged days in our run, with a calibration error of 0.0061 on its curve. The test asserts only that more than 200 days were judged and that the realised rate is within 0.03 of the budget; the two figures are printed, not asserted.

In part: `tests/monitor/test_wave7_demo.py::test_the_alert_volume_tracks_the_declared_budget` asserts a looser bound than the figures quoted.

9.3 Selection across many monitors

With ten thousand monitors each at $\alpha = 0.05$, five hundred false alarms a day is the expected outcome. PRAMA selects which p -values to raise with Benjamini–Hochberg [3], or with Benjamini–Yekutieli [4] when dependence is arbitrary, over a hierarchy domain $\rightarrow$ dataset $\rightarrow$ attribute $\rightarrow$ check; a parent’s p -value is combined from its children by Simes’ method, and a wholesale failure of most checks under one parent is rolled up into a single finding.

Runs: `tests/calibrate/test_select.py::test_bh_controls_the_false_discovery_rate`

Runs: `tests/calibrate/test_select.py::test_by_is_strictly_more_conservative_than_bh`

Runs: tests/calibrate/test_select.py::test_a_wholesale_failure_becomes_one_finding_rather_than_four_hundred

Not executed: a proof that the hierarchical procedure controls FDR at the declared level; the test measures flat BH, and the roll-up is a presentation rule, not part of the error guarantee.

Not in PRAMA: calibration on real financial data. Every figure in this section is from synthetic generators; the claim that the guarantees hold on a bank’s metric history is the central open question of this part of the work.

10 Metadata, correlation and fitness for purpose

Owners do not only declare structure. They write descriptions, tag fields (“mandatory”, “allowed values”, “PII”), and map attributes to glossary terms. Two further derivations use that material.

Definition 10.1 (Metadata-implied rule). A metadata template field of a given kind (flag, choice, list, number, ...) may carry a rule pattern with placeholders. Setting the field’s value on an attribute instantiates the pattern with the value as a *typed literal* and submits the result as a proposal with origin metadata. Changing the value retracts the earlier proposal and keeps its history.

Proposition 10.2 (A value cannot become syntax). *Instantiation quotes each value as a literal of the placeholder’s type, and a pattern may use only placeholders its field kind can fill. A value such as x') OR ($1=1$ is therefore a string literal in the resulting control, not a change to its predicate.*

Runs: tests/semantic/test_metadata.py::test_metadata_proposes_its_rules_and_retracts_them_when_changed

Runs: tests/semantic/test_metadata.py::test_a_value_cannot_inject_pql

Runs: tests/semantic/test_metadata.py::test_a_rule_may_only_use_placeholders_its_kind_can_fill

Definition 10.3 (Correlation). Attributes are grouped by shared meaning, trying in order a business concept, then a glossary term, then a semantic type; each attribute joins only its strongest group. Within a group, if exactly one dataset owns the value as a unique key, every other member is proposed a REFERENCES check to it. Generic semantic types (currency, dates, country, boolean) group for consistency only and never yield a reference.

Runs: tests/semantic/test_correlate.py::test_a_shared_term_groups_and_the_key_owner_is_referenced

Runs: tests/semantic/test_correlate.py::test_no_owner_or_two_owners_means_no_reference

Runs: tests/semantic/test_correlate.py::test_a_generic_type_groups_for_consistency_but_never_proposes_a_reference

Runs: tests/semantic/test_correlate.py::test_an_attribute_joins_only_its_strongest_group

In plain terms. If two columns in two datasets are both “the customer’s LEI”, and one dataset is the customer master where the LEI is unique, then every LEI elsewhere should exist in that master. If nobody, or two datasets, claim to own it, PRAMA proposes nothing, because it cannot tell which way the check should point.

Fitness search. A question in plain words — “trade amounts in USD”, “who is the customer and where are they registered, for sanctions screening” — is answered by ranking *datasets*, not rows, over their descriptions, metadata and attribute profiles: with Okapi BM25 [29] ($k_1 = 1.5$, $b = 0.75$) when no embedding model is configured, and with embeddings when one is. The result names the matched evidence (which attributes and fields matched) so that the ranking

can be checked. A discover-purpose model may re-rank and explain; it does not decide what exists.

Runs: `tests/semantic/test_fitness.py::test_without_an_embedding_model_bm25_ranks_and_names_its_evidence`

Runs: `tests/semantic/test_fitness.py::test_with_an_embedding_model_profiles_are_embedded_once_and_again_on_change`

Not in PRAMA: any retrieval-quality measurement (precision at k , nDCG) of fitness search; and correlation from data-side signals such as value overlap and query co-access.

Worked example: governance from metadata

Case study 7 declares three datasets and has the owners describe them: `customers.segment` allowed values `RETAIL`, `SME`, `CORPORATE` and mandatory; `payments.amount` minimum 0; the two LEI columns mapped to one glossary term; dataset keys. Nine rules follow as proposals, a reviewer accepts them, and correlation adds the reference from `accounts.customer_lei` to `customers.lei`, and also reports that the two columns sharing the term disagree on their PII flag. All four planted defects are found: six segments 'SME ' with a trailing space, four accounts with no LEI and three with an LEI no customer has (together the 7 integrity failures), and five negative payments. Asked “who is the customer and where are they registered, for sanctions screening”, fitness search returns *Customers*, naming `legal_name`, `customer_id` and `lei` as its evidence.

11 Evaluation

We report two kinds of evidence, and one absence. The case studies show the whole pipeline on planted defects, with both columns — what was planted and what was found — printed by the run. The benchmark measures bounds and ablations on a labelled corpus. Neither compares PRAMA with another system.

11.1 Eight case studies

Each study fabricates seeded banking data, declares an estate in business terms, lets Γ derive the controls, runs them, and prints the planted defects against the findings. The rule the studies follow is that every planted defect is stated before PRAMA is pointed at anything, in the generator and in the README, including the ones PRAMA will not find. We ran all eight (`python run.py -no-serve` in each study directory, against a scratch application database); Table 4 is transcribed from those runs.

What the table shows. Every defect a declared control can see was found, on the rows planted, with counts that match: 60 missing notionals, 7 negative quantities, 400 duplicated rows from a resent file, 37 orphan instruments, 18 settlement dates before trade dates, 31 half-cent rounding disagreements. The four “not established” outcomes are the screens of Section 4.4 (ISIN, LEI and a bank’s book-code check character) and the freshness control of Section 3; in none of them did PRAMA report a pass. The four “not claimed” are a relationship not declared in study 1 (two defects) and a relation between two dates nobody declared, and in study 2 a truncated file whose trailer check runs at arrival and is not yet wired into the control runner.

What the table does not show. The defects were planted by the author of the system that finds them, in data generated for the purpose, so the table demonstrates mechanism, not detection quality. Its most useful columns are the last two: a tool with no false negatives on data it was built against tells a reader nothing, while a tool that names what it did not catch,

Study	Planted	Found	Not est.	Not claimed	Controls
1 Trading book (SQLite)	9	4	2	3	65
2 Feeds: CSV, Parquet, JSON Lines	9	7	1	1	58
3 Mixed estate, relationships	4	4	0	0	59
4 Expressions and plugins	5	4	1	0	27
5 DQ delegates	4	4	0	0	14
6 Month-end close	4	3	0	0	18
7 Governance from metadata	4	4	0	0	10
8 From code to impact	2	2	0	0	7
Total	41	32	4	4	

Table 4: Planted defects against outcomes, from our runs of the eight case studies. *Not est.*: reported as not established (a screen, or a freshness control on a plain table). *Not claimed*: stated in the study as outside what the declared controls can see, with the reason. In study 6 the fourth planted item is a timing difference that is correctly *not* a break. *Controls* is the number executed on the main source; study 3 runs a further 22 on a second source.

and why, tells the reader what it is. The per-study counts are printed by each `run.py` and stated in each README; the test suite asserts only that each study runs to completion.

In part: `tests/casestudies/test_they_still_run.py::test_it_runs_to_completion` asserts exit status 0 for each study; the planted-versus-found counts are not asserted.

What building them found. Running the studies against data somebody had to look at found six defects in PRAMA, all of the kind this paper is about — artefacts that build, validate and look right while being wrong. `Γ` generated `MATCHES /None/` for every free-text column, because a value domain read back from JSON kept its kind as a string and the “is this constrained?” test passed for everything; controls were lowered without their code lists; a regular expression would not bind against a date column on a strict engine; SQLite was believed unable to do regular expressions; `ROUND` double-rounded on DuckDB; and an unknown function compiled straight to SQL while the reference interpreter returned unknown. Study 8 found two more, both fixed in the most recent revision: a lineage reconciliation proposed across a filter, which reported 118 cancelled trades as breaks, and a proposed reconciliation with no tolerance, which the engine refused to run.

11.2 The labelled defect benchmark

`prama bench run -seed 42` builds a corpus of 28 scenarios of 200 rows, one per defect class, across six families (structural, content, statistical, relational, temporal, semantic) and four difficulties (obvious 7, ordinary 9, subtle 11, adversarial 1), and labels only the rows that actually changed. A detection counts only when dataset, column and window all match. Our run:

Detector	Kind	Found	Precision	Recall	F1
detect-nothing	bound	0/28	–	0.00	–
alert-on-everything	bound	28/28	0.08	1.00	0.14
schema-only	ablation	2/28	0.67	0.07	0.13
patterns-only	ablation	5/28	0.83	0.18	0.29
statistics-only	ablation	7/28	0.88	0.25	0.39

The blind spots matter more than the F1: statistics-only found nothing in the relational family, and patterns-only nothing in the statistical, relational, temporal or semantic families. A

detector’s blind spots are the argument for whatever covers them, and they do not appear in an aggregate score.

Runs: `tests/bench/test_corpus.py::TestReproducibility::test_the_same_seed_gives_the_same_corpus`

Runs: `tests/bench/test_corpus.py::TestTheLabelsFollowWhatHappened::test_a_no_op_injection_plants_no_label`

Runs: `tests/bench/test_baselines.py::TestTheBoundsAreTheAxes::test_detecting_nothing_has_no_precision_rather_than_zero`

Runs: `tests/bench/test_baselines.py::TestTheAblationsAnswerWhichClaimDoesTheWork::test_no_ablation_comes_near_perfect_recall`

Runs: `tests/bench/test_scoring.py::TestTheMatchIsExact::test_the_wrong_column_does_not`

Runs: `tests/cli/test_bench_cli.py::TestRun::test_it_names_the_baselines_that_were_not_run`

The absence. None of the five detectors is PRAMA. The benchmark does not yet score PRAMA’s own controls and monitors on the corpus, and none of the fifteen external systems it names (Great Expectations, Soda Core, Deequ, dbt tests, HoloClean/Raha/Baran/ZeroED and others) was run; the command prints them as not run. We regard publishing the bounds and ablations without the comparison as better than publishing a comparison configured by the party that benefits from it, but it means this paper makes *no* claim about PRAMA’s detection quality relative to anything.

Not in PRAMA: PRAMA scored on its own benchmark; any external baseline; the banking-scale corpus; a blind evaluation with stewards (the blinding mechanism exists: `tests/bench/test_shadow.py::TestTheBlinding::test_what_a_steward_sees_carries_no_system`).

11.3 Measurements of the system itself

The following were measured in our runs and concern PRAMA’s own overhead, not detection. The test packages cited in this paper (1,693 tests across `backend`, `evidence`, `recon`, `derive`, `lineage`, `architecture`, `calibrate`, `score`, `llm`, `induce`, `propose`, `semantic`, `codeintake`, `delegates` and `bench`) passed, with one strict expected failure (the freshness control of Section 3), in 25 seconds. Fusion ran 400 generated controls in 160 scans. The backend conformance suite passed on DuckDB, SQLite, the reference interpreter and PostgreSQL 16.

12 The claims register

A paper that argues that beliefs about data must be justified owes the same to its own claims. The register below lists every formal claim with its state: **runs** (a named test exercises it), **in part** (something weaker runs, as stated at the claim), **stated** (mathematics or a specification that constrains the system and that the system does not execute), and **not built**.

Claim	State	Where, or why not
<i>Declarations and Γ</i>		
Relationships, thirteen kinds, never auto-confirmed (Definition 3.2)	runs	<code>test_services.py</code>
Determinism and identity stability (Proposition 3.5)	runs	<code>test_generator.py</code>
Provenance (Proposition 3.6)	runs	<code>test_generator.py</code>
Round trip (Proposition 3.7)	runs	<code>test_generator.py</code>
Coalescing (Proposition 3.8)	runs	<code>test_generator.py</code>
Refusal, not omission (Proposition 3.9)	runs	<code>test_relationships.py</code>

Claim	State	Where, or why not
Honest and flattering coverage (Proposition 3.11)	runs	<code>test_coverage.py</code>
Accuracy needs a relationship (Proposition 3.12)	runs	<code>test_wave6_acceptance.py</code> ; 82/71% figures prose
Only a declaration may skip review (Proposition 3.14)	runs	<code>test_queue.py</code> , <code>test_provenance.py</code>
A person activates every control (Section 3.5)	in part	one activation path, by inspection; untested
Corroboration merges (Proposition 3.16)	runs	<code>test_queue.py</code> , <code>test_utility.py</code>
A rhythm yields a freshness verdict	not built	strict xfail
PQL parity and roll-forward forms; hierarchy cycles	not built	comparison specifications only
<i>PQL, plans and verdicts</i>		
Plan identity closes over code lists and validators (Proposition 4.2)	in part	exclusions untested
Nothing scanned is not a pass (Proposition 4.4)	runs	<code>test_two_stage.py</code>
Unknown default is conservative (Proposition 4.6)	runs	<code>test_parser.py</code> , conformance
Screen soundness (Proposition 4.8)	runs	<code>test_two_stage.py</code> , <code>test_control_</code> <code>run.py</code>
Residual runs on the pushdown path	in part	reference interpreter only
Cross-engine agreement (Proposition 4.10)	runs	25 cases; 250 generated; PostgreSQL when configured
Semantic equivalence of compiler and interpreter	stated	tested, not proved
Refusal is conformance (Proposition 4.12)	in part	RE2 refusal untested
Fusion preserves answers and saves scans (Proposition 4.14)	runs	<code>test_fuse.py</code> , <code>test_throughput.py</code>
Custom SQL is one read-only query	runs	<code>test_custom_sql.py</code>
A delegate cannot decide (Proposition 4.16)	runs	<code>test_delegates.py</code>
Delegate sandbox isolation	in part	no network namespace; limits untested
Exact arithmetic	in part	operators exact; thresholds float
<i>Evidence</i>		
Tamper evidence (Theorem 5.2)	runs	<code>test_ledger.py</code> , <code>test_</code> <code>independent_verifier.py</code>
External anchoring of chain heads	not built	HMAC head; Ed25519 on bundles only
Merkle promotion (Definition 5.3)	runs	<code>test_ledger.py</code>
Independent verification (Proposition 5.5)	runs	<code>test_independent_verifier.py</code>
Erasures keeps the chain (tombstones)	runs	<code>test_independent_verifier.py</code>
Mixed format versions verify	runs	<code>test_format_versions.py</code>
Records under 2 KB	runs	<code>test_ledger.py</code> ; 848 bytes is prose
Replay classifies (Proposition 5.7)	runs	<code>test_replay.py</code> , <code>test_end_to_</code> <code>end.py</code>
<i>Reconciliation</i>		
Tolerance laws (Proposition 6.3)	runs	<code>test_reconciliation.py</code>
Exact match first (Lemma 6.5)	runs	<code>test_reconciliation.py</code>
Price of ignoring timing (Proposition 6.6)	in part	case study 6 run; not asserted
Break classes (Definition 6.7)	runs	<code>test_reconciliation.py</code>
A filter on each side of RECONCILE	runs	<code>test_reconcile_pql.py</code> , <code>test_</code> <code>joins.py</code>
CLASSIFY on RECONCILE; signed certificate	not built	
<i>Lineage</i>		
Parsed, never executed; fallback recorded	runs	<code>test_pyspark.py</code> , <code>test_sql.py</code> , <code>test_intake.py</code>
Gold-fixture precision and recall	runs	<code>test_gold_fixture.py</code>

Claim	State	Where, or why not
Strongest path (Proposition 7.3)	runs	<code>test_graph.py</code>
Attenuation constants calibrated	stated	declared constants, not estimates
What a copy justifies (Proposition 7.5)	runs	<code>test_lineage_controls.py</code> , <code>test_reconcile_pql.py</code>
Join predicates invisible (Proposition 7.6)	in part	case study 8 run; the repair is asserted
Population-dependence edges	in part	SQL reader, <code>test_joins.py</code> ; PySpark records a gap
Every driving row finds its match (join checks)	runs	<code>test_reconcile_pql.py</code>
<i>Models</i>		
Architectural guard (Proposition 8.2)	runs	<code>test_layering.py</code> , with counterfactual guard is syntactic
Model-free verdict path, as information flow	stated	
No vacuous control reaches a reviewer (Proposition 8.4)	runs	<code>test_validate.py</code> , <code>test_inducer.py</code>
Hash-chained call ledger (Definition 8.5)	in part	chain tested; CLI untested
Evaluation gates activation (Proposition 8.6)	runs	<code>test_governance.py</code>
Residency, redaction, budgets	runs	<code>test_providers.py</code> , <code>test_gateway.py</code>
Quality of model-authored rules	not built	not measured
<i>Trust and calibration</i>		
Trust choices disagree (Definition 9.1)	runs	<code>test_trust.py</code>
Semiring laws of the trust choices	stated	only one is a semiring
Conformal validity (Theorem 9.4)	runs	<code>test_conformal.py</code>
Unexpressible budgets refused (Corollary 9.5)	runs	<code>test_select.py</code>
Drift mechanisms and their costs	runs	<code>test_conformal.py</code>
Some mechanism calibrated per regime (Table 3)	runs	<code>test_benchmark.py</code>
Declared budget realised end to end	in part	looser bound asserted than quoted
BH and BY control FDR	runs	<code>test_select.py</code>
Hierarchical FDR guarantee	stated	roll-up is presentation
Calibration on real financial data	not built	synthetic only
<i>Metadata and search</i>		
A value cannot become syntax (Proposition 10.2)	runs	<code>test_metadata.py</code>
Correlation and key ownership (Definition 10.3)	runs	<code>test_correlate.py</code>
Fitness search names its evidence	runs	<code>test_fitness.py</code>
Retrieval quality of fitness search	not built	not measured
<i>Evaluation</i>		
Benchmark bounds and ablations	runs	<code>tests/bench</code>
PRAMA scored on the benchmark; external baselines	not built	named, not run
Case studies: planted against found (Table 4)	in part	exit status asserted; counts printed

Of seventy-two rows, forty-seven run, twelve run in part, five are stated without being executed, and eight are not built.

The ratio is not flattering and is not meant to be. The claims that run are concentrated where the thesis lives: derivation, verdict semantics and evidence. The claims that do not are

concentrated where a claim is about the world rather than about the code — detection quality against other systems, calibration on real data, whether reviewers accept what models propose — and those need a bank, a steward and time, none of which a test suite can supply.

13 Related work

Rule frameworks and validation. Deequ [30] and TFX data validation [6] made declarative assertions over large datasets practical, and open frameworks in the same family (Great Expectations, Soda, dbt tests) are widely deployed. They take the assertion as the primitive. PRAMA takes the *declaration* as the primitive and derives assertions, which is what makes coverage measurable and a control’s existence explainable. Data contracts [5] move in the same direction, though contract tooling frequently records obligations without enforcing them [7]; PRAMA imports ODCS quality blocks where they state a rule it has, and refuses the rest by name. SHACL [31, 32] is the closest standard to a declaration-first model for graph data.

Discovery and cleaning. Profiling and dependency discovery [25, 26, 18, 11] find constraints the data satisfies; error detection and repair [28, 21, 35] find cells that are probably wrong. Both are sources of *proposals* in PRAMA, ranked below a declaration because a column that happens to be unique today is not a declared key. Example-driven induction follows the label-efficient line of Raha and Baran [21] and weak supervision [27].

Models as rule authors. Recent work uses language models to generate data quality rules [19, 20, 33]. Our contribution here is a boundary rather than a generator: the counterfactual gate (Proposition 8.4), the evaluation gate on activation (Proposition 8.6), and a structural argument that a model can influence a verdict only through a person’s activation.

Calibration and multiple testing. Conformal prediction [34] gives distribution-free false-alarm guarantees under exchangeability; adaptive conformal inference [14] trades per-test validity for long-run rate control under drift; conformal anomaly detection with FDR control is an active area [12, 23]. Benjamini–Hochberg [3] and Benjamini–Yekutieli [4] control the false discovery rate across tests. We apply these off the shelf; the contribution is honest degradation (refusing a budget the history cannot express) and reporting a grid rather than a number.

Provenance and lineage. Provenance semirings [15] give the algebra of which our trust choices are instances, or near-instances (Section 9). OpenLineage [24] standardises the exchange of lineage events; PRAMA emits it and reads SQL, Python and BI models itself, and our negative result (Proposition 7.6) concerns any lineage model that records value flow only.

Tamper-evident records. Hash-linked timestamping [16], Merkle trees [22], tamper-evident logging [8] and certificate transparency [17] are the foundations of the evidence ledger. The ledger’s contribution is modest and practical: a verifier that shares no code with the writer, held equal to it by a test. It lacks the external anchoring these systems use.

Regulation and practice. BCBS 239 [2] frames the obligation; fines for reporting errors [1] price it; market surveys [13, 9, 10] describe the fragmented tooling that the thesis responds to.

14 Limitations and how this could be wrong

Do owners declare? The thesis rests on business owners writing declarations. The case studies declare on the owners’ behalf. Nothing here measures whether a real steward, in a real

week, produces the declarations that make Γ useful. If they do not, derivation has nothing to derive from, and the system falls back to mining and induction — the proposals this paper ranks lowest.

Planted defects are not defects. Every defect in the evaluation was planted by the author, and the benchmark does not yet score PRAMA. The paper therefore makes no claim of superior detection. What it claims is narrower: that a verdict PRAMA reports is justified in the sense of Section 1.3, and that what it cannot justify it reports as not established.

Synthetic calibration. The conformal guarantees are theorems under exchangeability, and the drift mechanisms are measured on generators the author wrote. A bank’s metric history may violate the assumptions in ways none of the five regimes represents.

Scale. The declaration model and Γ have not been exercised on an estate of thousands of datasets; the throughput figures measure PRAMA’s own overhead against an in-memory engine, not a warehouse.

Connectors and engines. Conformance runs on DuckDB, SQLite and PostgreSQL. Other dialects reached through JDBC are code-complete but not verified against live services, and a Snowflake connector has never met an account; the repository says so wherever it describes them.

The guard is syntactic. Proposition 8.2 is a necessary condition checked by co-occurrence of names. It would not detect a model’s output passed through neutral names across modules. The structural argument — a verdict is a function of an activated plan and the data — is the stronger defence, and it is argued, not checked.

Evidence trusts its head. Without external anchoring, a party holding the HMAC key can rewrite a chain and re-sign its head. Tamper evidence is against everyone else.

What would falsify the thesis. A declared estate on which derived controls miss defects that hand-written controls catch, at comparable effort; a deployment in which *indeterminate* is so frequent that operators treat it as a pass; or a replay that reports *identical* for a run whose data changed. Each is a measurable outcome, and the first two need real users.

15 Conclusion

A data quality claim is a belief, and a belief is worth acting on when it is justified. We turned that into three constraints — controls derived from declarations, verdicts from a deterministic engine, evidence that verifies without its author — and showed what each requires: a total, provenance-carrying map from declarations to controls with a coverage measure that does not flatter; a verdict set in which “the SQL found nothing” can be reported as what it is; and a record an auditor can check with the standard library. On that base, reconciliation acquires a tolerance semantics and a timing law, lineage acquires a precise statement of which controls a copy justifies, and models acquire a boundary they cannot cross except through a person.

The register counts forty-seven claims that run and twenty-five that do not, or do so only in part. The ones that run are the ones the thesis depends on. The ones that do not are, for the most part, claims about people and data that the author does not own, and they are where the work goes next.

References

- [1] EMIR and MiFID transaction reporting challenges continue to trouble firms and regulators. A-Team Insight. FCA fines: about £95M MiFID transaction reporting, about £34.5M EMIR; Goldman Sachs >£34M for 220M+ errors; UBS >£27M for 135M+ errors. URL: <https://a-teaminsight.com/blog/emir-and-mifid-transaction-reporting-challenges-continue-to-trouble-firms-and-regulators/>.
- [2] Basel Committee on Banking Supervision. Principles for effective risk data aggregation and risk reporting (BCBS 239), 2013.
- [3] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate. *JRSS-B*, 57(1), 1995.
- [4] Yoav Benjamini and Daniel Yekutieli. The control of the false discovery rate in multiple testing under dependency. *Annals of Statistics*, 2001.
- [5] Bitol Project, Linux Foundation. Open data contract standard (odcs) v3.1.0, December 2025. URL: <https://bitol-io.github.io/open-data-contract-standard/v3.1.0/>.
- [6] Eric Breck, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data validation for machine learning (TFX), 2019.
- [7] Most data contract tools don’t enforce contracts: Here’s what does, 2026. URL: <https://zircote.com/blog/2026/04/most-data-contract-tools-dont-enforce-contracts/>.
- [8] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium*, 2009.
- [9] The 2026 data quality and data observability commercial software landscape. DataKitchen, 2026. URL: <https://datakitchen.io/blog/the-2026-data-quality-and-data-observability-commercial-software-landscape/>.
- [10] A comparison of data quality frameworks: A review. *Big Data and Cognitive Computing*, 9(4), 2025. URL: <https://www.mdpi.com/2504-2289/9/4/93>.
- [11] Anna Fariha, Ashish Tiwari, Arjun Radhakrishna, Sumit Gulwani, and Alexandra Meliou. Conformance constraint discovery: Measuring trust in data-driven systems. 2020. URL: <https://arxiv.org/pdf/2003.01289>.
- [12] Adaptive novelty detection with false discovery rate guarantee, 2022. URL: <https://arxiv.org/pdf/2208.06685>.
- [13] Magic quadrant for augmented data quality solutions, March 2025. URL: <https://www.gartner.com/en/documents/6246519>.
- [14] Isaac Gibbs and Emmanuel Candès. Adaptive conformal inference under distribution shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.
- [15] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the 26th ACM Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007.
- [16] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991.

- [17] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, Internet Engineering Task Force, 2013.
- [18] Ester Livshits, Alireza Heidari, Ihab F. Ilyas, and Benny Kimelfeld. Discovery of approximate (and exact) denial constraints. *PVLDB*, 13(3), 2019. URL: <https://dl.acm.org/doi/10.14778/3368289.3368293>.
- [19] LLM-DQR: Large language model-based automated generation of data quality rules for electronic health records. *Journal of Biomedical Informatics*, 2025. URL: <https://www.sciencedirect.com/science/article/abs/pii/S1532046425001807>.
- [20] Quality assessment of tabular data using large language models and code generation, 2025. URL: <https://arxiv.org/abs/2509.10572>.
- [21] Mohammad Mahdavi and Ziawasch Abedjan. Semi-supervised data cleaning with raha and baran. In *CIDR*, 2021. URL: https://vldb.org/cidrdb/papers/2021/cidr2021_paper14.pdf.
- [22] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, volume 293 of *LNCS*, pages 369–378. Springer, 1988.
- [23] Conformal anomaly detection in python (nonconform), 2026. URL: <https://arxiv.org/pdf/2605.13642>.
- [24] Openlineage: An open standard for data lineage, 2024.
- [25] Thorsten Papenbrock, Tanja Bergmann, Moritz Finke, Jakob Zwiener, and Felix Naumann. Data profiling with metanome. In *PVLDB*, 2015.
- [26] Thorsten Papenbrock et al. Functional dependency discovery: An experimental evaluation of seven algorithms. *PVLDB*, 2015.
- [27] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. In *PVLDB*, 2017. URL: <https://arxiv.org/pdf/1711.10160>.
- [28] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. HoloClean: Holistic data repairs with probabilistic inference. In *PVLDB*, 2017.
- [29] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [30] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. Automating large-scale data quality verification (deequ), 2018.
- [31] Is SHACL suitable for data quality assessment?, 2025. URL: <https://arxiv.org/html/2507.22305v1>.
- [32] A review of SHACL: From data validation to schema reasoning for RDF graphs, 2021. URL: <https://arxiv.org/pdf/2112.01441>.
- [33] Template-guided rule generation and evaluation for data quality using large language models, 2025. URL: <https://repositum.tuwien.at/handle/20.500.12708/221852>.
- [34] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

- [35] ZeroED: Hybrid zero-shot error detection through large language model reasoning, 2025.
URL: <https://arxiv.org/pdf/2504.05345>.